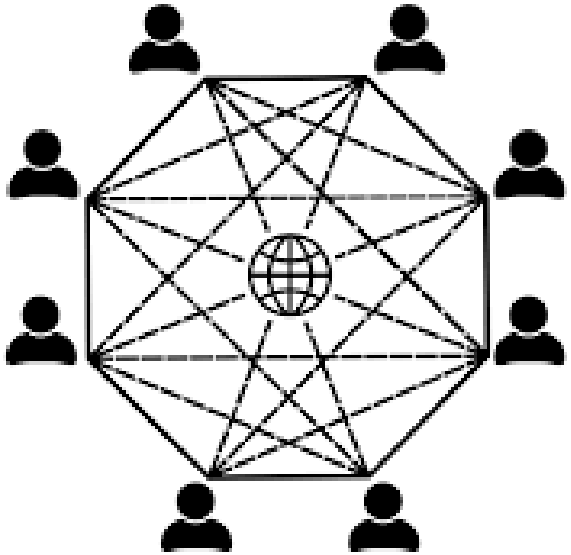
네트워크설계

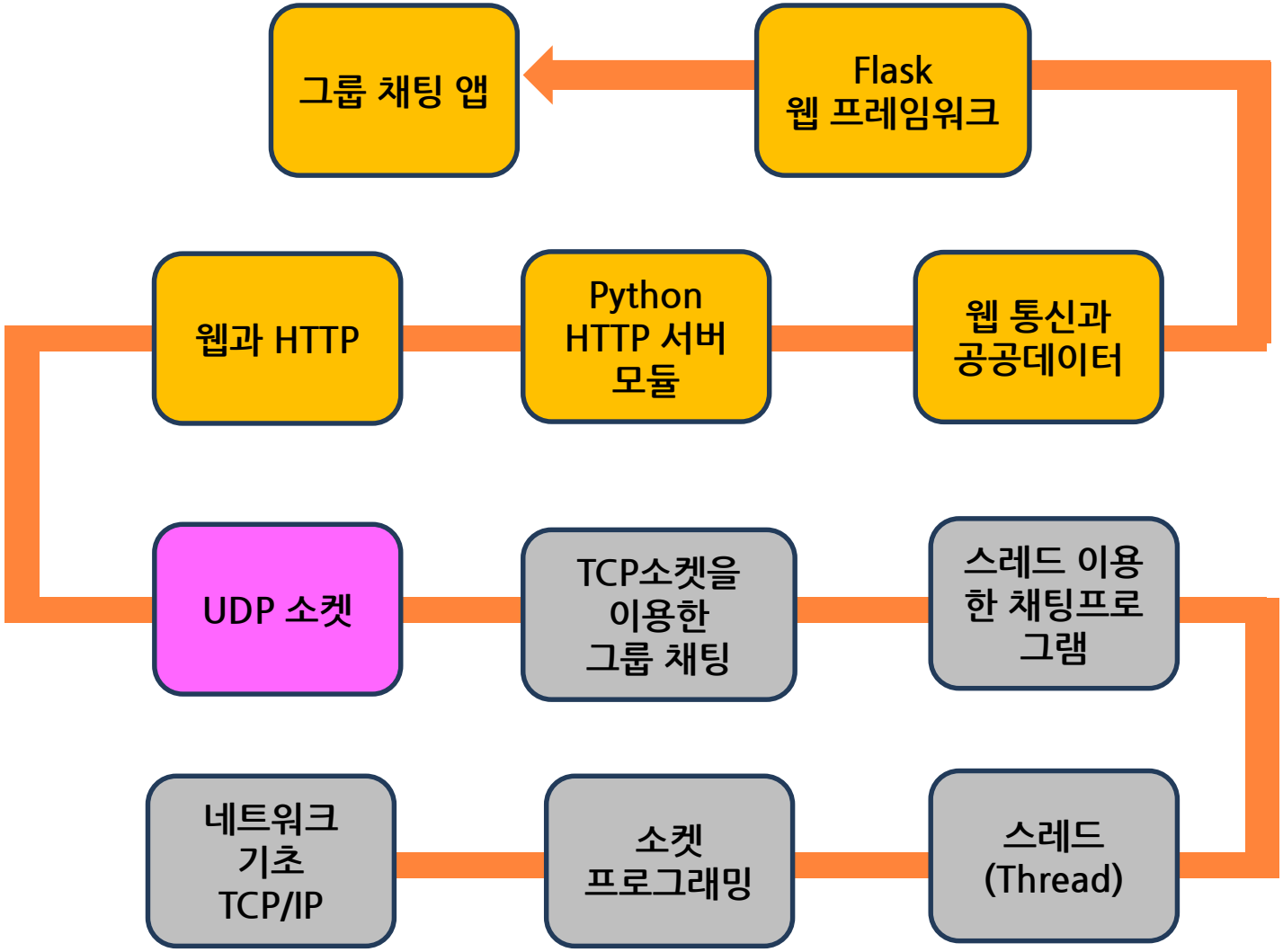
7주차 - UDP 소켓통신



인공지능학과

그림참조: https://pxhere.com/ko/photo/1637436

7주차는 UDP 소켓 통신



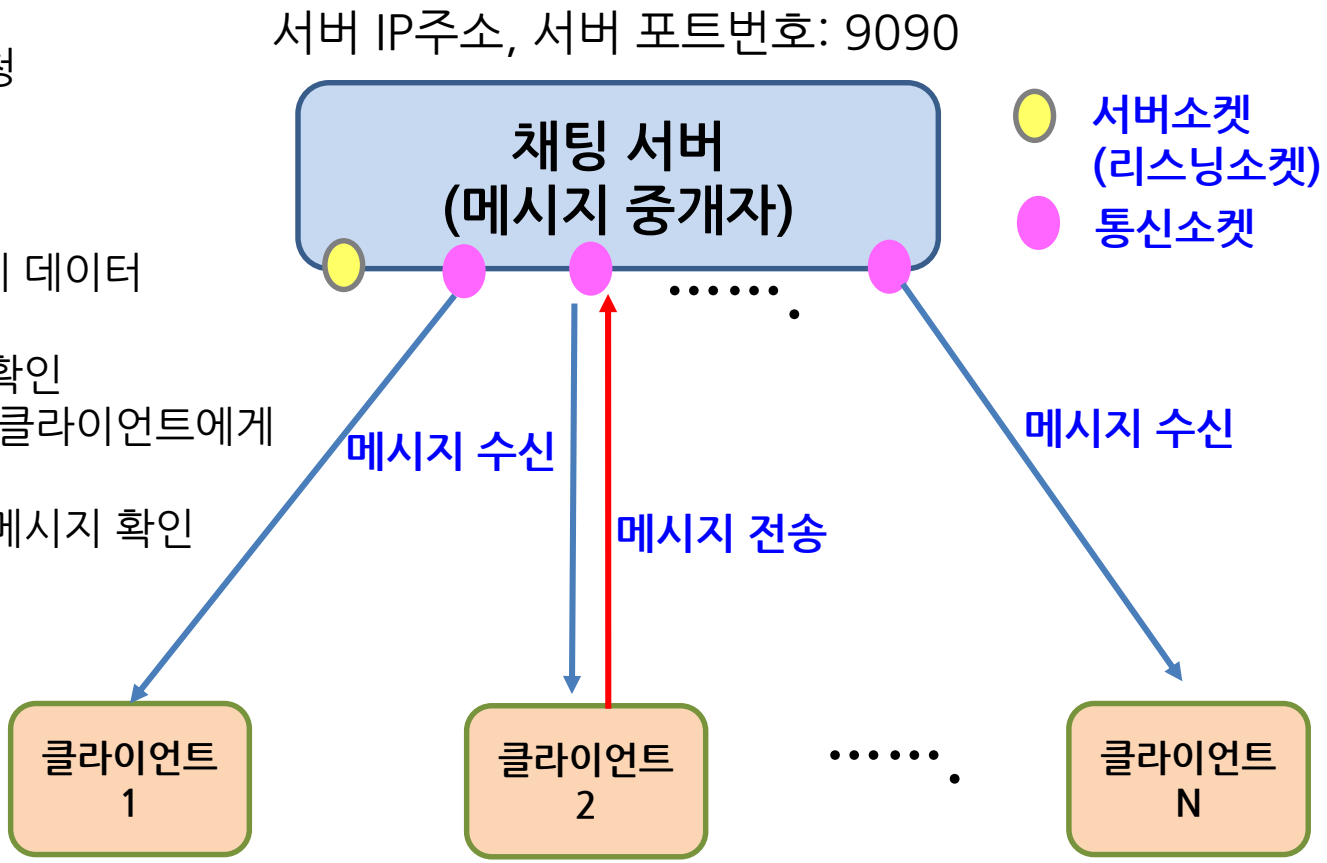
앱 제작 실습 병행

5주차정리. TCP소켓과 스레드를 이용한 그룹채팅

- **TCP 소켓과 스레드를 이용해서 그룹 채팅 프로그래밍**

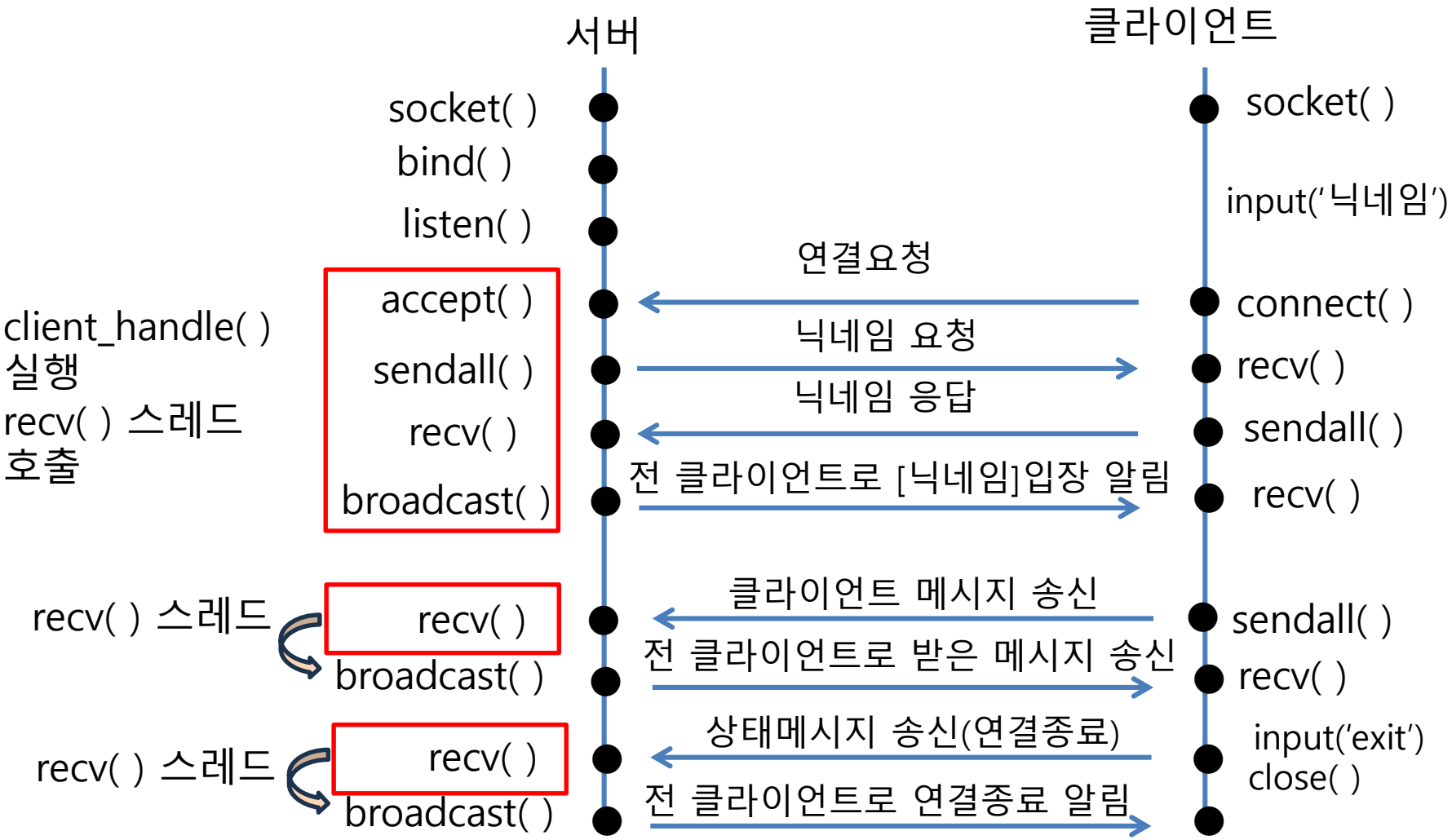
  - **그룹 채팅방은 이렇게 동작합니다**

1. 채팅 서버 동작
2. 클라이언트 TCP 접속 요청
3. 서버는 클라이언트에게 닉네임 요청
4. 클라이언트 닉네임 전송
5. 특정 클라이언트가 서버에 데이터 전송
6. 서버는 전달받은 메시지 확인
7. 서버에 접속해 있는 모든 클라이언트에게 받은 메시지 전송
8. 모든 클라이언트는 같은 메시지 확인



5주차정리. TCP소켓과 스레드를 이용한 그룹채팅

- **그룹 채팅방 메시지 전달 과정**



5주차정리. TCP소켓과 스레드를 이용한 그룹채팅 - 서버

▪ 그룹채팅방 서버 프로그램(1)

- 메시지를 보낸 클라이언트는 제외하고 모든 클라이언트에게 보내기

```python
# TCP Group chatting server program w/ thread

import socket, threading

# 클라이언트 소켓과 닉네임을 저장할 리스트 준비
clients = []  # 클라이언트 소켓객체를 저장할 리스트
nicknames = [] # 클라이언트 닉네임을 저장할 리스트

# 메시지 수신되면 대화방에 참여한 전 클라이언트에게 전송
def broadcast(message, sender):
    for client in clients:
        if client != sender:
            client.sendall(message.encode('utf-8'))

```

```python
# 클라이언트 수신된 메시지 처리
def recv(client):
    while True:
        try: # 정상적으로 메시지 수신되면
             # 수신된 메시지 broadcast로 전달
            rxmsg = client.recv(1024).decode()
            broadcast(rxmsg, client)
        except: # 네트워크 오류 발생시
            index = clients.index(client)
            clients.remove(client) # 클라이언트 정보삭제
            client.close()   # 클라이언트 소켓 종료
            nickname = nicknames[index]
            broadcast(f"{nickname}님이 퇴장했습니다", None)
            nicknames.remove(nickname) # 닉네임 삭제
            print(f"{nickname}퇴장, {len(clients)}명 참가중")
            break

```

▪ 그룹채팅방 서버 프로그램(2)

```
32 # 대화방에 참여하는 클라이언트 관리
33 def client_handle():
34     while True:
35         conn, addr = server.accept()
36         print(f">> {addr} is connected")
37         # 접속된 클라이언트에세 닉네임 요청 및 저장
38         conn.sendall('NICK'.encode('utf-8'))
39         nickname = conn.recv(1024).decode('utf-8')
40         print(f"{nickname} 입장")  # 서버용 출력
41         nicknames.append(nickname)
42         clients.append(conn)
43         # 새로 접속된 클라이언트 정보를 모든 클라이언트에게 알람
44         broadcast(f"{nickname}님이 입장했습니다", None)
45         broadcast(f"{len(clients)}명이 대화방에 참가중입니다", None)
46         conn.sendall(f"{nickname}님 반갑습니다".encode('utf-8'))
47         # 새로 참가한 클라이언트의 수신 스레드 생성 및 실행
48         receiver = threading.Thread(target=recv, args=(conn,))
49         receiver.start()
50
```

그룹채팅방 서버 프로그램(3)

```
51 if __name__ == "__main__":
52     IP = '0.0.0.0'  # 서버 IP
53     PORT = 9090
54     address = (IP, PORT)  # tuple로 주소정보 저장
55
56     print(">> Group Chatting Server start")
57     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
58     # (optional code) winError 10048 에러 해결을 위한 추가 코드
59     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)
60     server.bind(address)
61     server.listen()
62     client_handle()
```

■ 그룹채팅방 클라이언트 프로그램(1)

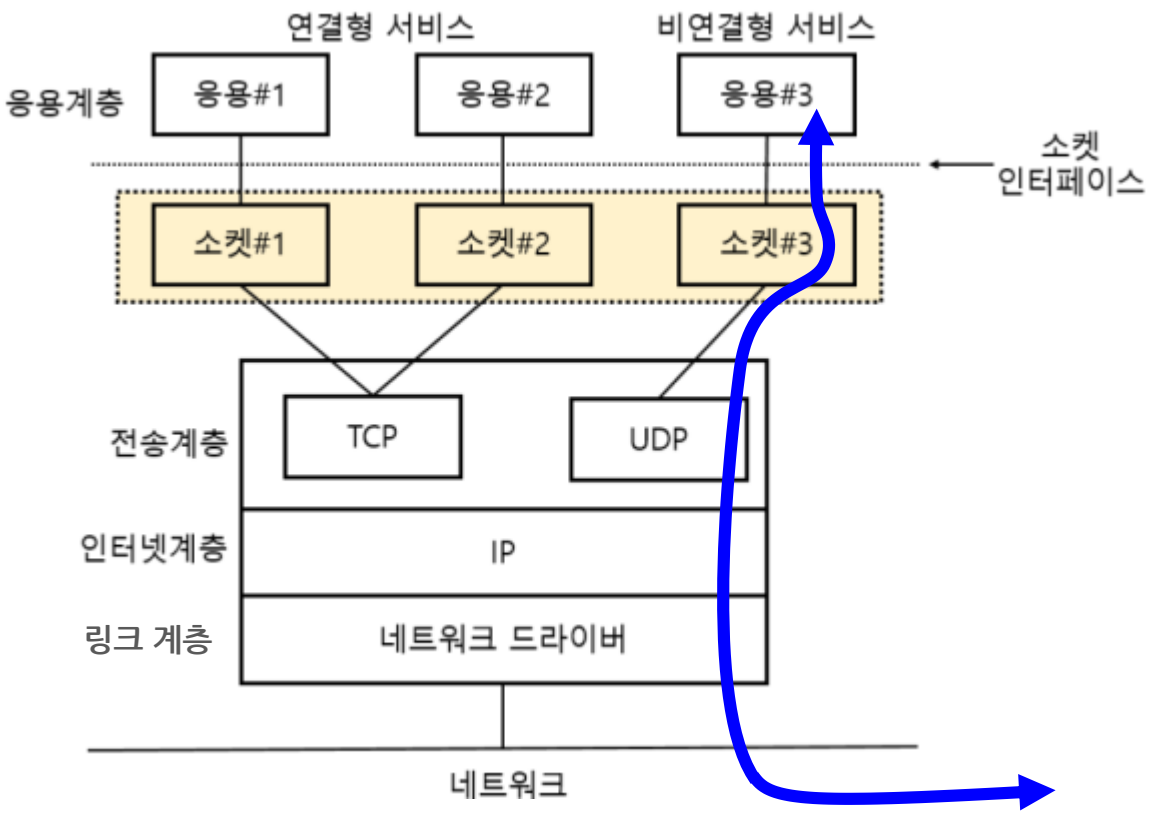
파일명: tcpGroupChatClient.py

```python
# TCP Group chatting client program w/ thread
import socket, threading

def send(client):
    while True:
        message = input("Send >>")
        sendmsg = f"[{nickname}]: {message}"
        client.sendall(sendmsg.encode('utf-8'))
        if message == 'exit':
            print(">> Exit by User")
            client.close() # 소켓 종료
            break

def recv(client):
    while True:
        try:
            # 서버로 수신된 메시지가 'NICK'인 경우 닉네임 전송
            rxmsg = client.recv(1024).decode('utf-8')
            if rxmsg == 'NICK':
                client.sendall(nickname.encode('utf-8'))
            else: # 채팅 메시지인 경우 출력
                print(rxmsg)
        except: #네트워크 오류인 경우 소켓종료
            print("Network error occurred!")
            client.close()
            break
```

▪ 그룹채팅방 클라이언트 프로그램(2)

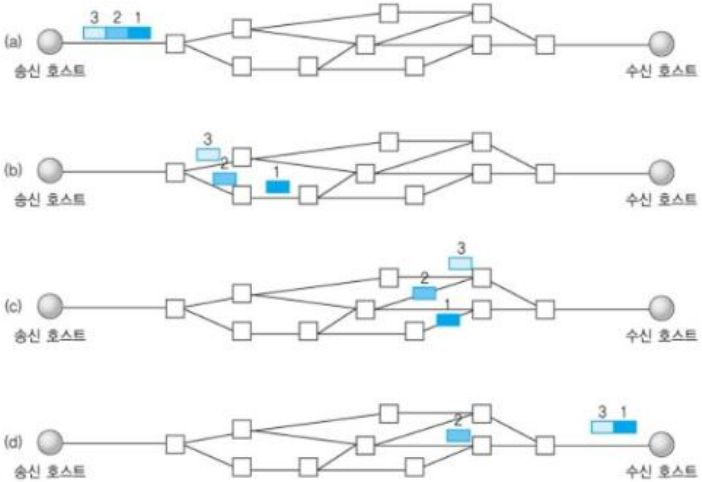
```
27
28 if __name__ == "__main__":
29     IP = '10.2.40.241'  # 서버 IP
30     PORT = 9090         # 서버 포트번호
31     address = (IP, PORT)  # tuple로 주소정보 저장
32
33     # 대화방에 사용할 닉네임 정하기
34     nickname = input("Your nickname:")
35
36     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
37     client.connect(address)
38
39     # 메시지 송신과 수신을 스레드로 생성하고 실행
40     receiver = threading.Thread(target=recv, args=(client,))
41     receiver.start()
42     sender = threading.Thread(target=send, args=(client,))
43     sender.start()
44     while True:
45         receiver.join()
46         sender.join()
47         break
48     print("Client End")
```

UDP 소켓 통신


연결형 서비스
비연결형 서비스
응용계층
응용#1
응용#2
응용#3
소켓
인터페이스
소켓#1
소켓#2
소켓#3
전송계층
TCP
UDP
인터넷계층
IP
링크 계층
네트워크 드라이버
네트워크

1. UDP 소켓 통신

▪ UDP(User Datagram Protocol) 특징



[UDP 특징]

- 비연결형 서비스이고 신뢰성이 낮다
- 정보를 주고 받을 때 정보를 보내거나 받는다는 신호절차를 거치지 않는다.
- 최소한의 오류만 검출한다.
- TCP보다 속도가 빠르다

[UDP 서버의 특징]

- UDP에는 연결 자체가 없어서(connect 함수 불필요)
  서버(리스닝) 소켓과 통신 소켓의 구분이 없다.
- 소켓 대신 IP를 기반으로 데이터를 전송한다.
- 서버와 클라이언트는 1대1, 1대N, N대M 등으로 연결될 수 있다.
- 신뢰성이 필요한 서비스보다 성능이 중요시 되는 경우에 사용된다.

1. UDP 소켓 통신

- TCP와 UDP 비교

프로토콜 종류	TCP	UDP
연결 방식	연결형 서비스	비연결형 서비스
패킷 교환 방식	가상 회선 방식	데이터그램 방식
전송 순서	전송 순서 보장	전송 순서가 바뀔 수 있음
수신 여부 확인	수신 여부를 확인함	수신 여부를 확인하지 않음
통신 방식	1:1 통신	1:1 OR 1:N or N:N 통신
신뢰성	높다	낮다
속도	느리다	빠르다

1. UDP 소켓 통신

▪ UDP 소켓 서버/클라이언트 특징

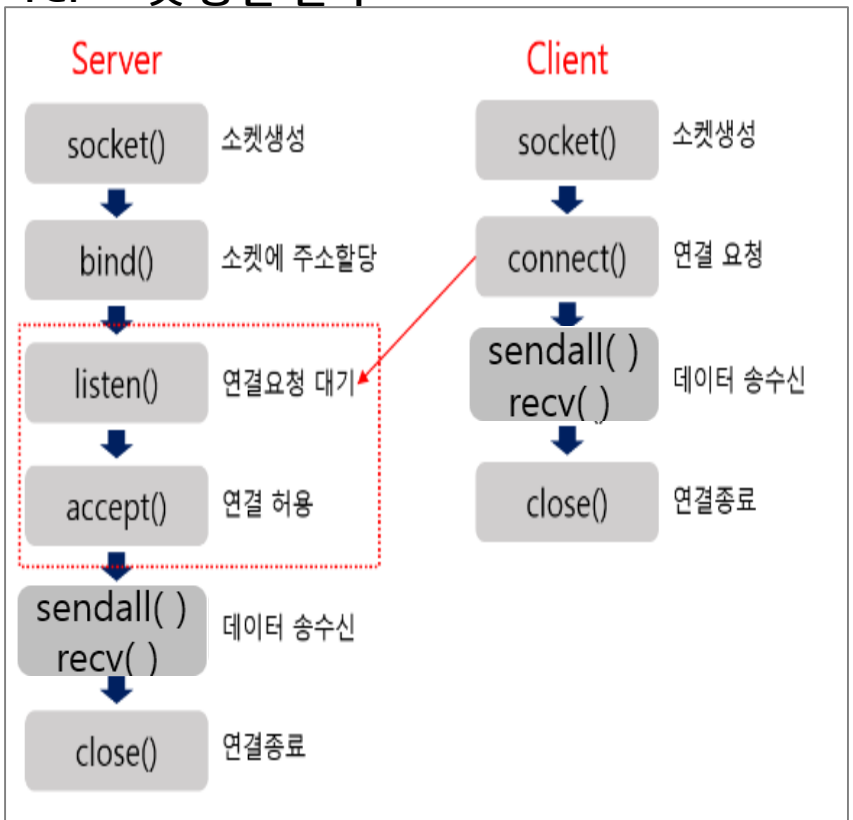
1) UDP 에서의 서버와 클라이언트는 연결되어있지 않다.

- TCP와 달리 연결 설정의 과정이 필요 없다.

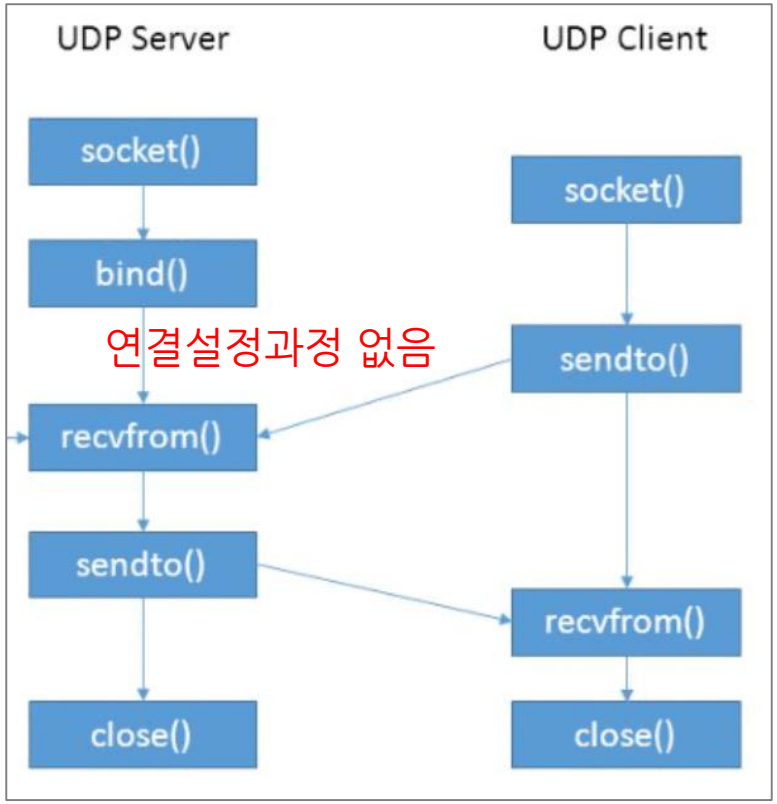
  따라서 TCP 서버 구현과정에서 거쳤던 listen 함수와 accept 함수의 호출은 없다

  다만 UDP 소켓의 생성과 데이터의 송수신 과정만 존재

TCP 소켓 통신 절차



UDP 소켓 통신 절차



1. UDP 소켓 통신

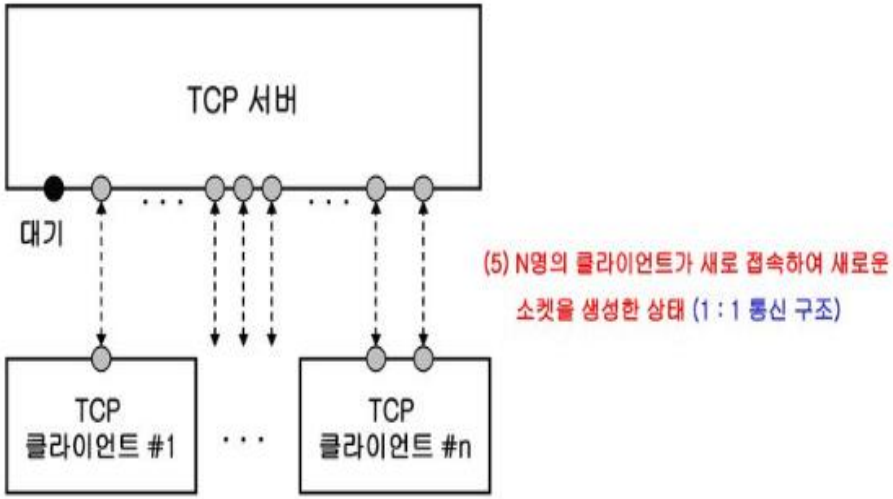
▪ UDP 소켓 서버/클라이언트 특징

2) UDP에서는 서버와 클라이언트 모두 하나의 소켓만 있으면 된다.

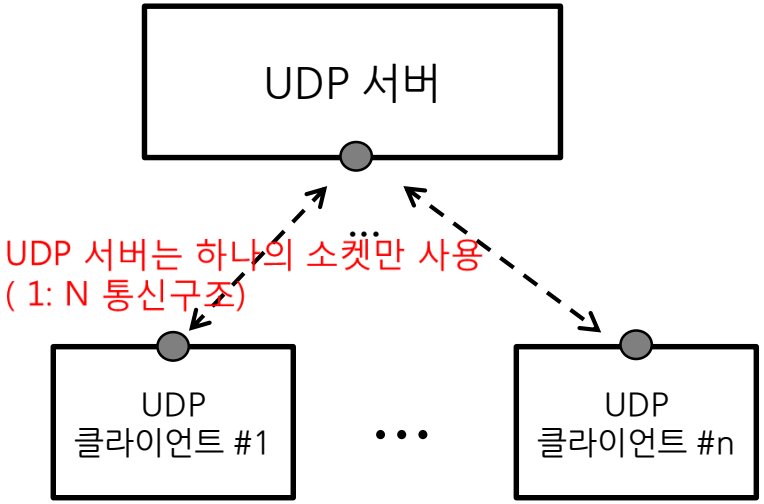
예) 10개의 클라이언트와 통신하기 위해서

- TCP에서는 소켓과 소켓의 관계가 1:1이었다.
  때문에 서버에서 열 개의 클라이언트에게 서비스를 제공하려면
  서버(리스닝) 소켓 하나와 열 개의 통신소켓이 더 필요
- 그러나 UDP는 서버건 클라이언트건 하나의 소켓만 있으면 된다

TCP 소켓 구조



UDP 소켓 구조



UDP 소켓 서버/클라이언트 구현

3) UDP 소켓에서 데이터 전송 시 상대방의 주소정보 별도 추가

- **TCP 소켓**을 생성하고 나서 데이터를 전송하는 경우,
  데이터 전송 시 상대방의 주소 정보를 따로 추가하는 과정이 필요 없다.
  (TCP 소켓은 연결설정을 통해 이미 목적지의 주소정보를 이미 알고 있는 상태이기 때문)

TCP

전송
```
client.sendall("Hello Server".encode('utf-8'))
```

수신
```
rxmsg = conn.recv(1024)
```

- 그러나 **UDP 소켓**은 연결상태를 유지하지 않으므로
  데이터를 전송할 때마다 반드시 목적지의 주소 정보를 별도로 추가해야 한다.
  수신 시에는 return 값이 두 개 (수신메시지와 상대방의 주소 정보)

상대방 목적지의 IP주소와 포트정보 추가

UDP

전송
```
client.sendto("Hello Server".encode(), ('10.2.100.34', 8888))
```

수신 시 return값이 두 개 (수신메시지, 상대방의 IP주소와 포트정보)

수신
```
data, addr = server.recvfrom(1024)
```

1. UDP 소켓 통신

- **UDP 소켓 서버 함수**

서버소켓 생성

server = socket.**socket**(socket.AF_INET, socket.SOCK_DGRAM)

Family = IPv4 Type = UDP

서버소켓 바인딩

server.**bind** ((**서버IP주소**, **PORT번호**)) #튜플로 저장

서버 소켓에서 메시지 수신

data, addr = server.**recvfrom** (1024) **# BUFFER_SIZE = 1024**

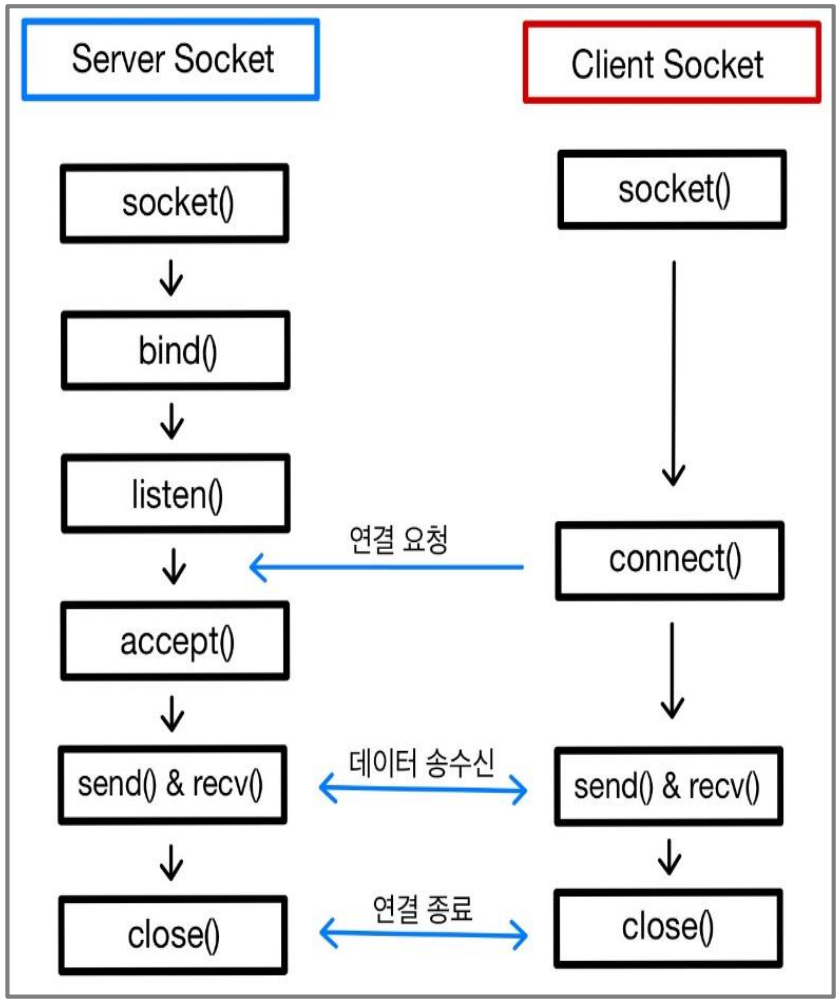
상대방의 IP주소와 포트정보

서버 소켓에서 메시지 전송

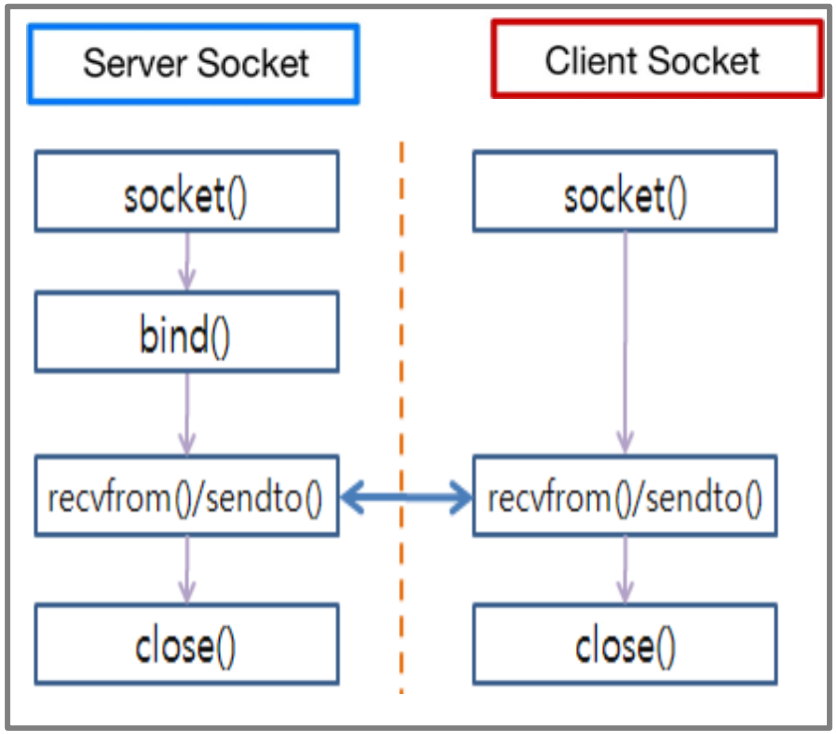
server.**sendto** (data.encode( ), (**목적지IP주소**, **목적지PORT번호**))

TCP 소켓과 UDP 소켓의 통신 절차 비교

TCP 소켓통신 절차



UDP 소켓통신 절차



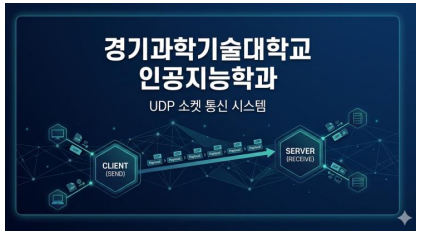
- 연결설정과정이 없음
- 서버는 UDP 소켓 1개만 사용
- 데이터 전송 시 상대방의 주소정보 추가
- 데이터 수신 시 상대방의 주소정보 반환

2. UDP 소켓을 이용한 파일 전송 실습1

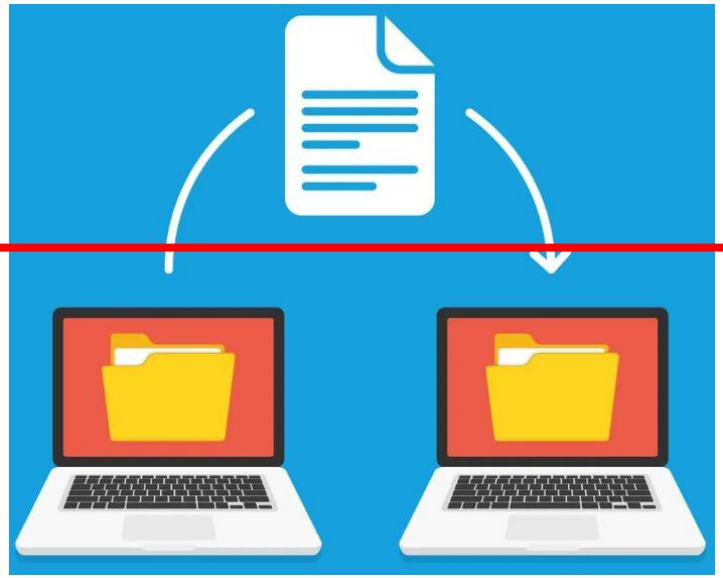
▪ UDP 소켓을 이용한 파일 전송

- UDP는 안정성이 떨어지나 데이터를 더 빠른 속도로 전송하는 데 사용하므로 오디오 및 비디오 파일과 같은 데이터를 전송하는 데 사용

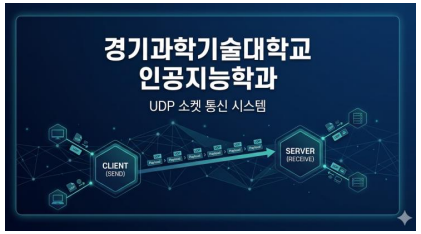
" UDP 소켓을 통해 텍스트가 아닌 이미지 파일을 송수신해 봅시다! "



intro.png



서버 클라이언트



intro.png

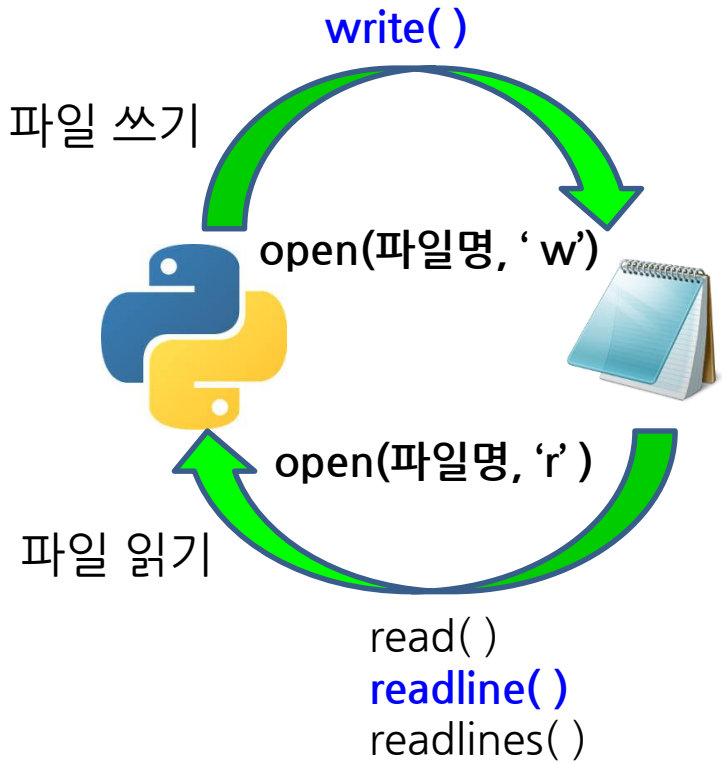
- 서버에서 이미지파일(intro_ai.png) 파일을 클라이언트로 전송하기
- 클라이언트에서는 파일 수신 후 화면에 출력하여 정상 수신 확인하기

2. UDP 소켓을 이용한 파일 전송 실습1

- **파이썬 파일 입출력 하는 방법 : open( ), read( ), write( ), close( )**

  - “text.txt” 파일에 문자열 입력하기

  - “text.txt” 파일 읽어서 출력하기




```
# 파일 입출력하기
# 쓰기모드로 파일을 열어서 데이터 저장하기
# open() : 파일을 열고 파일 객체를 반환하는 함수 (read/write)
f = open('text.txt', 'w', encoding='utf-8')
f.write("홍길동님\n")
f.write("안녕하세요\n")
f.write("반갑습니다\n")
f.close()  # 파일 닫기

# 읽기모드로 파일을 열어서 데이터 읽기
f = open('text.txt', 'r', encoding='utf-8')
data = f.read()  # 파일전체를 문자열로 리턴
print(data)
f.close()  # 파일닫기
```

```
홍길동님
안녕하세요
반갑습니다
```

read( ): 파일 전체 내용을 한번에 읽어서 저장
readline( ) :파일의 내용을 한 줄씩 읽어서 저장
readlines( ): 파일의 내용을 모두 읽어 리스트에 저장

2. UDP 소켓을 이용한 파일 전송 실습1

- 실습1) UDP 소켓을 이용한 파일 전송

실습 순서)

1) 전송할 이미지파일 다운(intro.png) 받아 파이썬 프로젝트 폴드에 저장
   (*주피터 노트북 사용자도 역시 코드가 실행되는 같은 폴더에 저장)

2) UDP 소켓을 이용해 파일 전송하는 서버 프로그램 제작

3) UDP 소켓을 이용해 파일을 수신하는 클라이언트 프로그램 제작
   (*파이썬 이미지 처리 라이브러리인 Pillow 가 설치되어 있지 않은 경우 설치. 다음장 참조)

4) 서버 프로그램 실행

5) 클라이언트 프로그램 실행 (클라이언트 이름(닉네임) 전송)

6) 서버에서 다운받아 놓은 'intro.png' 이미지 파일 전송

7) 클라이언트에서 파일 수신 후 화면에 출력하여 정상 수신 확인

8) 소켓 종료

참조) 파이썬에서 이미지 처리용 라이브러리: PIL(PILLOW)

PIL(PILLOW)는 이미지 분석 및 처리에 사용되는 파이썬 라이브러리

- 이미지 저장 및 불러오기
- 효과 주기 (크롭핑, 블러, 밝기 보정, 색상 보정 ...)
- 확장자 변경
- 기타 등등

※ PIL 라이브러리가 내 컴퓨터에 설치되어 있지 않은 경우

설치 방법

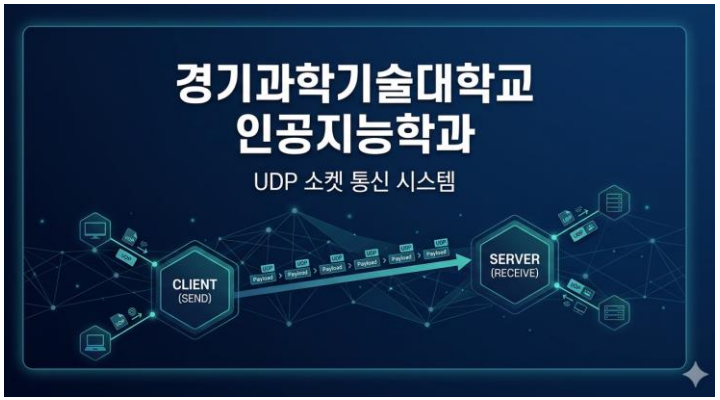
```
pip install pillow
```

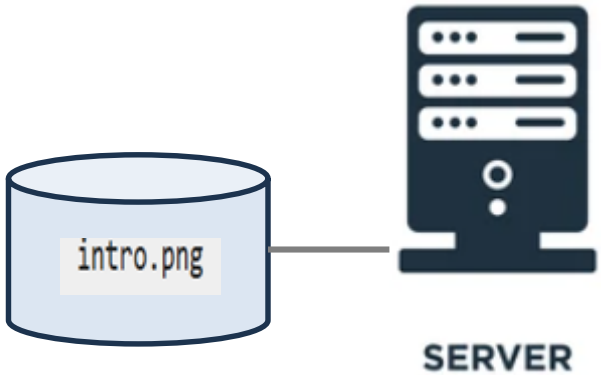
파이썬 터미널 또는 윈도우 명령프롬프트(cmd)에서 설치

2. UDP 소켓을 이용한 파일 전송 실습1

■ 실습1) UDP 소켓을 이용한 파일 전송

intro.png






```
file:  intro.png
>> UDP server start
홍길동 ('127.0.0.1', 50441) 님이 접속하였습니다
Sending intro.png .....
```

"닉네임" 전송

"file name" 전송

file data 전송

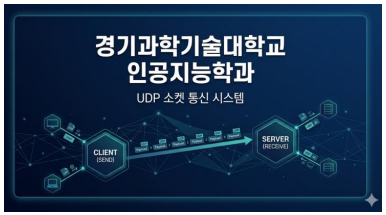
Client

```
>> UDP client start
Choose your nickname:  홍길동
>> file name:  intro_copy.png
.............................................
.............................................
.............................................
..................intro_copy.png Finish!
Image showing.......
```

2. UDP 소켓을 이용한 파일 전송 실습1

■ 실습1) UDP 소켓을 이용한 파일 전송 서버 프로그램 udpServer.py

```python
# UDP 소켓을 이용한 파일전송 서버 프로그램
import socket, time

IMAGE_FILE = 'intro.png'

IP = 'localhost'  # localloop ip
PORT = 8888
address = (IP, PORT)

file_name = IMAGE_FILE
print("file:", file_name)

print(">> UDP Server start")
# UDP 소켓 객체 생성
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# 소켓 binding: 소켓에 IP주소와 port번호 지정
server.bind(address)

```




```python
# recvfrom(): 클라이언트 메시지(닉네임) 수신
data, addr = server.recvfrom(1024)
print(f"{data.decode('utf-8')}, {addr}님이 접속하였습니다")

# sendto(data,addr): 클라이언트 파일이름 전송
server.sendto(file_name.encode('utf-8'), addr)
print(f"Sending {file_name} .....")

# 파일오픈, 이미지는 바이너리로 지정
f = open(file_name, 'rb')
data = f.read(1024)  # 1KB씩 읽어서 전송

while data:
    # 파일 전송, 바이너리인경우 인코딩 필요없음
    if server.sendto(data, addr):
        data = f.read(1024) # 1KB씩 읽어서 전송
        time.sleep(0.02) # 클라이언트 데이터 저장시간 보장

server.close()
f.close()
print(">> UDP Server End")
```

"닉네임" 수신

"file name" 전송

file data 전송

2. UDP 소켓을 이용한 파일 전송 실습1

■ 실습1) UDP 소켓을 이용한 파일 전송 클라이언트 프로그램

udpClient.py

```python
# UDP 소켓을 이용한 파일전송 클라이언트 프로그램
import socket, time
import os, select
from PIL import Image

IP = 'localhost'  # locall
PORT = 8888
address = (IP, PORT)
timeout = 3

print(">> UDP Client Start")
# UDP 소켓 객체 생성
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 닉네임 지정
nickname = input("Choose your nickname:")
# sendto() 닉네임 전송
client.sendto(nickname.encode('utf-8'), address)
```

※ select( )함수는 참조만!

```
import select
select.select(rlist, wlist, xlist[, timeout])
```

arguments:

rlist : 읽기 준비를 기다릴 파일 디스크립터 리스트. file, sockt 등.
wlist : 쓰기 준비를 기다릴 파일 디스크립터 리스트.
xlist : exception 상태를 기다릴 파일 디스트립터 리스트.
timeout : 대기 시간. 단위 초.

"닉네임" 전송

```python
# recvfrom() 데이터(파일명) 수신
data, addr = client.recvfrom(1024)
if data:
    file_name = data.decode('utf-8')
    # 디렉토리에 같은 파일명이 있는 경우 파일명 변경
    if os.path.exists(file_name):
        index = file_name.find('.')
        old_name = file_name[:index]
        file_extension = file_name[index:]
        file_name = old_name + '_copy' + file_extension
    print(">> file name:", file_name)
# 파일 오픈해서 수신된 이미지 저장, 이미지는 바이너리로 저장
f = open(file_name, 'wb')
while True: # 수신루프
    print('.', end='')
    # 소켓에 수신된 파일이 있는지 체크
    ready = select.select([client],[],[], timeout)
    if ready[0]: # 파일이 있는 경우
        data, addr = client.recvfrom(1024)
        f.write(data)
    else: # 파일이 다 수신된 경우
        print(f"{file_name} Finish!")
        f.close()
        # 수신된 이미지 파일을 화면에 출력하기
        img = Image.open(file_name)
        print("Image showing.......")
        img.show()
        break
client.close()
print(">> UDP client End")
```

"file name" 수신

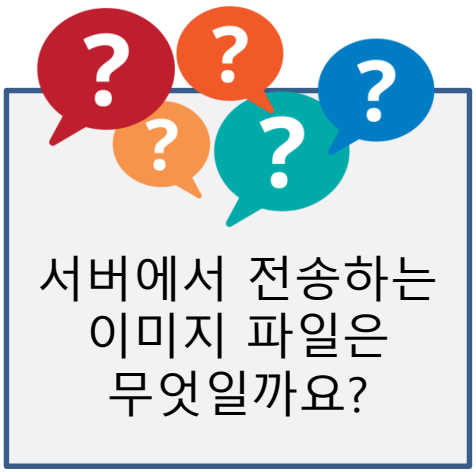
file data 수신

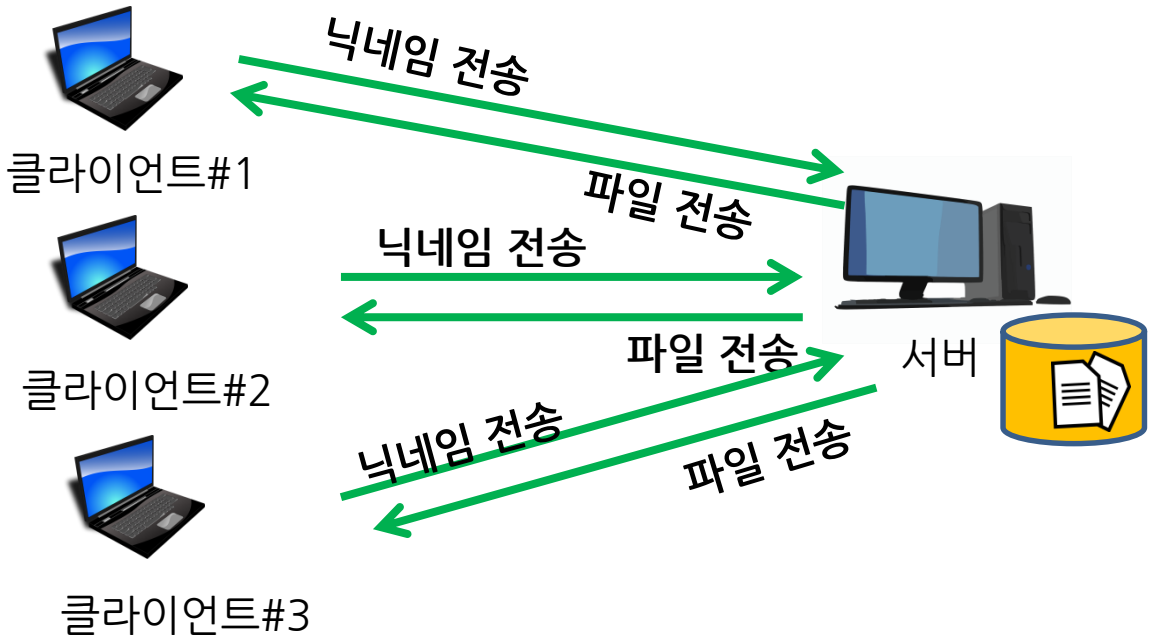
3. UDP 소켓을 이용한 파일 전송 실습2

- 실습2) 실습1을 통해 서버와 클라이언트간 성공적으로 파일전송이 확인된 경우

  UDP 소켓을 이용해 교수 컴퓨터(UDP 서버)에서 파일 전송 받기

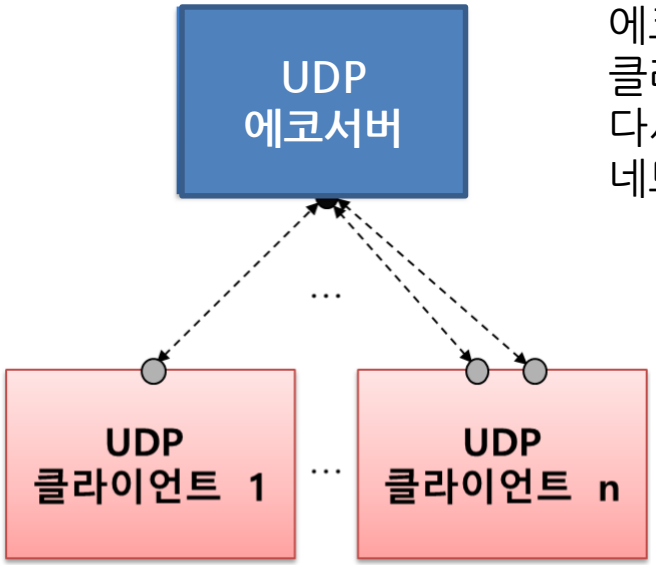
  - UDP 클라이언트 코드 사용
  - 교수 컴퓨터(UDP 서버)의 IP주소와 포트 번호는 수업시간에 지정하여 공지





  - 전송받은 이미지 파일은 과제로 제출

UDP 소켓을 이용한 에코서버와 클라이언트



에코서버란?
클라이언트가 보낸 데이터를 서버에서 받아 그대로 다시 클라이언트에게 돌려보내 주는(메아리) 기본적인 네트워크 서버

- 스레드를 사용하지 않고도, 한 소켓으로 여러 클라이언트 처리 가능
- 클라이언트는 별도의 연결설정 과정없이 서버와 통신
- UDP 서버는 하나의 소켓을 이용해 다수의 클라이언트와 통신 가능

4. UDP 소켓을 이용한 에코서버와 클라이언트

■ UDP 소켓을 이용한 에코서버와 클라이언트

```python
# UDP 소켓을 이용한 에코 서버 프로그램
import socket

IP = 'localhost'  # localloop ip
PORT = 9000
address = (IP, PORT)

print(">> UDP Echo Server start")
# UDP 소켓 객체 생성
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# 소켓 binding: 소켓에 IP주소와 port번호 지정
server.bind(address)

while True:
    # recvfrom(): 클라이언트 메시지 수신
    data, addr = server.recvfrom(1024)
    print(f"{addr}:{data.decode('utf-8')}")

    # sendto(data,addr): 클라이언트가 보낸메시지 그대로 전송
    server.sendto(data, addr)

server.close()
print(">> UDP Server End")
```

```python
# UDP 소켓을 이용한 에코서버 클라이언트 프로그램
import socket

IP = 'localhost'  # localloop IP
PORT = 9000
address = (IP, PORT)

print(">> UDP Echo Client Start")
# UDP 소켓 객체 생성
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
    # 보낼메시지 입력
    sendmsg = input("Send:")
    # sendto() 서버로 메시지 보내기
    if sendmsg == 'exit':
        print("Disconnect by User")
        break
    client.sendto(sendmsg.encode('utf-8'), address)
    # recvfrom() 서버로부터 되돌아 온 메시지 수신
    data, addr = client.recvfrom(1024)
    print("Recv:", data.decode('utf-8'))

client.close()
print(">> UDP client End")
```

7주차 실습 과제

- **과제명: UDP 소켓 실습 파일**

  - 제출과제:
    1) 파일서버로 부터 받은 이미지 파일
    2) UDP 에코서버와 클라이언트 파이썬 파일
    PC에 저장 후 과제 제출란에 제출
  - 첨부파일: **이름.png (파일서버로 부터 받은 이미지 파일)**
    udpEchoServer_이름.py , udpEchoClinet_이름.py

  - 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.

네트워크 설계 중간고사 안내

❑ **시험일자: 4/27(월) , 수업시간**

❑ 시험범위
- 수업시간에 다룬 내용만 출제되며, 강의자료 위주로 공부하세요.
- 강의자료는 온라인학습사이트(ctl.gtec.ac.kr)에서 주차별 교안을 다운받으면 됩니다.

No	주제	주요내용	비중
1	네트워크 TCP/IP 기초	- TCP/IP 4계층 구조, 프로토콜과 인터페이스 - TCP와 UDP 차이 이해 - 소켓 개념과 IP주소, 포트번호 이해	대략 40%
2	TCP 소켓 절차와 TCP 소켓 프로그래밍	- TCP 소켓 사용절차 - TCP 소켓을 이용한 에코서버와 클라이언트 (부분코딩)	대략 30%
3	스레드를 이용한 TCP 채팅 프로그래밍	- 스레드의 기본개념과 사용법 (생성, 실행, join()) - 스레드와 TCP 소켓을 이용한 1:1 (부분코딩)	대략 15%
4	UDP 소켓 절차와 UDP 소켓 프로그래밍	- UDP 소켓 특징과 함수 사용절차 (TCP 소켓과의 차이) - UDP 소켓 을 이용한 에코서버와 클라이언트(부분코딩)	대략 15%

※ 객관식(단답형포함)과 주관식
※ 부분코딩문제: 1) TCP 소켓을 이용한 에코서버와 클라이언트 (함수 사용위주)
2) 스레드와 TCP 소켓을 이용한 1:1 채팅 (스레드 사용법 위주)
3) UDP 소켓을 이용한 에코서버와 클라이언트 (함수 사용위주)
※ 용어설명

실습 내용 요약

2~7주차까지 실습한 내용 중 핵심 사항 위주로 정리한 자료입니다.

중간고사 시험 대비하여

주차별 핵심 내용은 꼭 이해하시기 바랍니다.

공부하다 이해가 안 되는 부분이 있으면 아래 연락처로 문의해 주세요.

연락처) 010-2345-5243

E-mail) ickim1@gtec.ac.kr

연구실) C동 311호

TCP/IP 4계층 구조

- 네트워크는 계층구조로 이루어짐

- TCP/IP 4계층 구조 : 응용 - 전송 - 인터넷(네트워크) - 링크

TCP/IP 4계층

① 응용 계층
② 전송 계층
③ 인터넷 계층
④ 링크 계층

① **응용 계층**
사용자 어플리케이션에서 사용하는 프로토콜
예) FTP, HTTP, SSH, Telnet, DNS, SMTP 등

② **전송계층**
- 통신 노드 간의 연결 제어 및 자료 송수신을 담당
- TCP(Transmission Control Protocol) : 연결형, 신뢰성
- UDP(User Datagram Protocol): 비연결형, 비신뢰성

③ 인터넷(**네트워크) 계층**
- 최종 목적지까지 정확하게 연결되도록 경로 설정: 라우팅
- Host를 구분하기 위해 주소를 할당: IP 주소
- IP(Internet Protocol) : 호스트들과 네트워크에서 주소 관리, 패킷을 라우팅

④ **링크 계층**
- 물리적으로 데이터가 네트워크를 통해 어떻게 전송되는지를 정의, 물리주소(MAC주소)를 사용
예) Ethernet, WiFi등

프로토콜과 인터페이스

- 프로토콜: 서로 다른 호스트에 위치한 동일 계층끼리의 통신 규칙
- 인터페이스: 같은 호스트에 위치한 상하위 계층 사이의 규칙
- 서비스: 하위 계층이 상위 계층에 제공하는 인터페이스

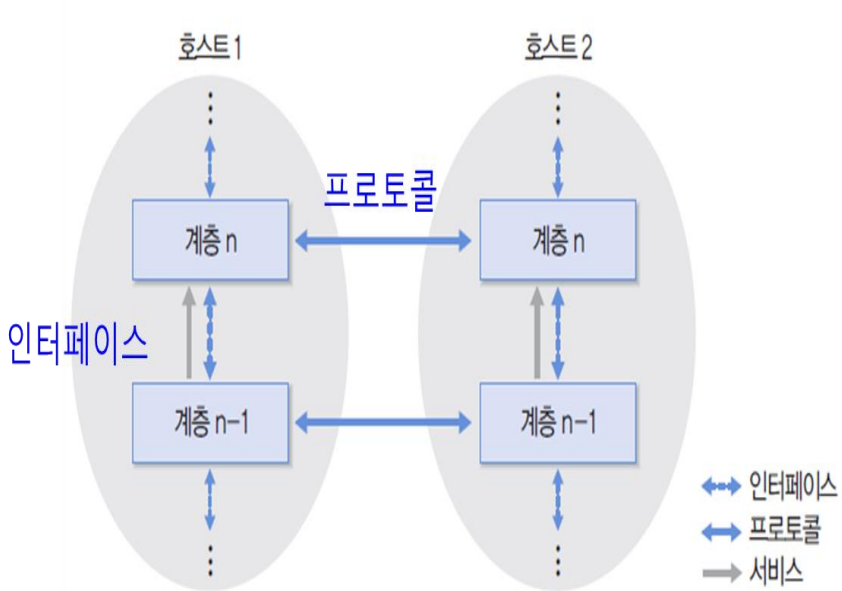
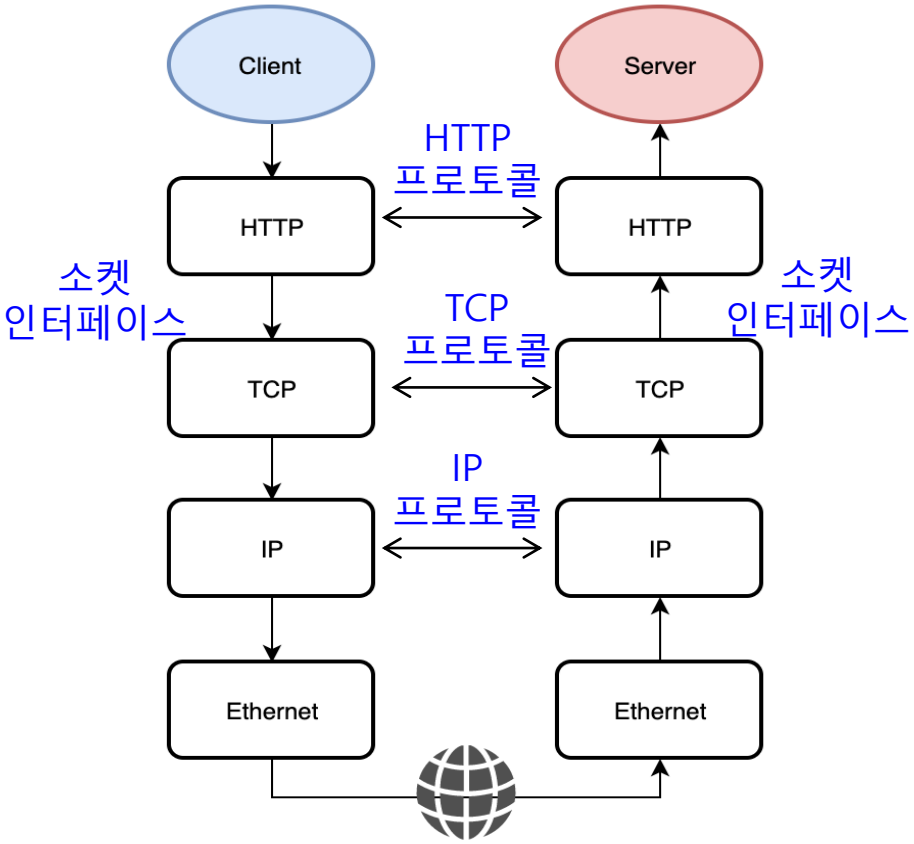
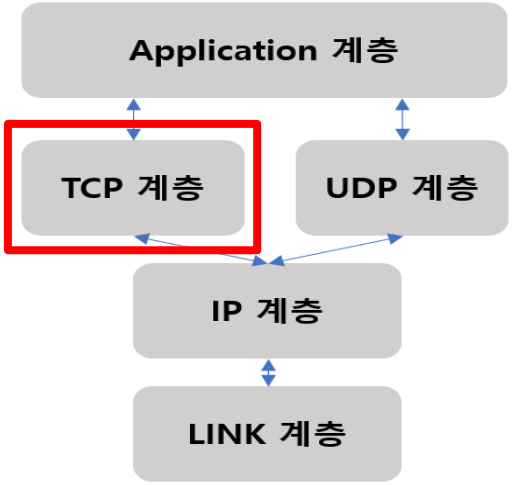


그림 1-5 인터페이스와 프로토콜



TCP(Transfer Control Protocol)



- **연결형**이며, **신뢰성**있는 데이터 통신을 가능하게 해주는 전송계층의 프로토콜

① **연결 지향적**(Connection-Oriented)

- 데이터를 전송하기 전에 먼저 수신 측과 연결을 설정

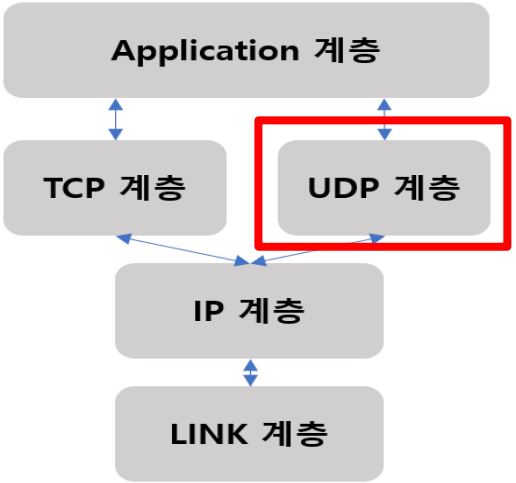
② **신뢰성 있는 데이터 통신**

- TCP는 송신 측과 수신 측 사이의 흐름제어 / 오류제어 / 혼잡제어 기능 제공

- 전송되는 데이터의 양을 조절하는 **흐름 제어**
- 데이터가 유실되거나 잘못된 데이터가 수신되었을 경우 대처하는 방법인 **오류 제어**
- 네트워크 혼잡에 대처하는 **혼잡 제어**

→ **응용계층은 전송계층의** TCP 덕분에 예외 상황에 크게 신경 쓰지 않고 응용계층 동작에 집중할 수 있음

UDP(User Datagram Protocol)



- **비연결형, 비신뢰성 서비스를 제공하는 전송계층 프로토콜**

- 흐름제어 기능이 없어 버퍼 오버플로Buffer Overflow에 의한 데이터 분실오류가 발생할 수 있음
- 에러제어 기능은 없으며 최소한의 오류검출 기능만 제공
  (패킷에러가 검출되면 폐기: 재전송하지 않음)

- **TCP보다 속도가 빠르다**
- 음성 및 비디오를 위한 실시간 스트리밍 서비스에 주로 사용 (그 외 DNS, DHCP등)

1. 네트워크 TCP/IP 기초 - 소켓 개념

■ 네트워크 소켓

- 응용 프로그램간(서버와 클라이언트) 네트워크를 통해 서로 통신할 수 있도록 연결해 주는 소프트웨어(라이브러리)
- 응용계층과 전송계층 사이에 위치, 응용프로그램이 TCP/IP를 이용할 수 있도록 인터페이스 역할
- 소켓에는 IP 주소와 Port번호가 할당 (IP주소로 컴퓨터를 특정, Port번호로 프로세스를 특정)

응용계층

소켓

전송계층

인터넷계층

응용 프로그램 | 응용 프로그램 | 응용 프로그램 | 응용 프로그램

TCP 소켓 | TCP 소켓 | UDP 소켓 | UDP 소켓

Port 번호 | Port 번호

TCP | UDP

IP

IP주소: 10.2.40.45

IP 주소 (IPv4 주소)

- 네트워크상에서 **컴퓨터(Host)를 식별하기 위해 부여된 주소**
- **32bit(4byte)** 길이로 구성된 논리적인 주소체계 (2^{32} 개 IP주소 배정가능)
- IP 주소는 **네트워크 주소**와 **호스트 주소** 두 부분으로 나뉜다. (IP주소 배정 규칙)

IP 주소 = Network ID + Host ID

	Network ID			Host ID
IPv4 주소	**192 .**	**168 .**	**10.**	**10**
서브넷 마스크	**255.**	**255.**	**255.**	**0**
	11111111	**11111111**	**11111111**	**00000000**

** 서브넷 마스크(AND 연산) 은 IP주소에서 네트워크 부분과 호스트 부분을 나누는 역할*

- Network(네트워크) ID : 네트워크 주소 (Prefix)
- Host(호스트) ID: 하나의 네트워크 내에 존재하는 호스트를 구분하기 위한 주소

IP주소의 종류

종류	특징
공인 IP (Public IP)	• **전세계적으로 중복되지 않는 유일한 주소** • 인터넷 서비스 제공업체(ISP)가 사용자에게 할당하는 주소로 **인터넷 가능**
사설 IP (Private IP)	• **IP주소 부족문제를 해결하기 위해 등장** • 일반 가정이나 회사 등의 내부 네트워크에 할당된 IP 주소 • **사설 IP 주소만으로는 인터넷에 직접 연결할 수 없음** • 내부에서 외부로는 접근이 가능하지만 외부에서는 내부로 접근이 불가능 • IPv4 사설 IP 대역 - **10**.0.0.0 ~ **10**.255.255.255 (10.0.0.0/8) - **172.16.**0.0 ~ 172.31.255.255 (172.16.0.0/12) - **192.168**.0.0 ~ 192.168.255.255 (192.168.0.0/16)
루프백 IP (localhost)	• 네트워크상에서 자신을 나타내는 가상적인 주소 • **외부 네트워크를 사용하지 않고, 로컬에서 소프트웨어를 테스트하는 데 사용** • 대표적인 루프백 IP는 127.0.0.1 (localhost)를 사용
브로드캐스트 IP	• **네트워크에 있는 컴퓨터나 장비 모두에게 한 번에 데이터를 전송하는데 사용** • 255.255.255.255 또는 • IP 주소의 SUBNET 부분을 제외한 호스트 자리를 2진수 1로 채운 것
모든 IP	• 0.0.0.0**(모든 IP 주소)'는 내 컴퓨터에 연결된 모든 주소**

1. 네트워크 TCP/IP 기초 - 포트번호

■ 포트번호(Port number)

- IP주소를 통해 패킷 메시지가 목적지 컴퓨터까지 왔다면 이제 목적지 컴퓨터에서 어떤 프로그램(Application)에게 가야 하는지 알려주는 것이 포트번호 (Port number)
  - 포트번호: 0 ~ 65535

- Port 번호는 3가지로 구분

1) 널리 알려진 포트 (Well Known Port) : 0 ~ 1023

*어떤 특권을 가진 서비스에 사용되는 포트(시스템포트)
사용자가 직접 등록해서 사용할 수 없는 포트

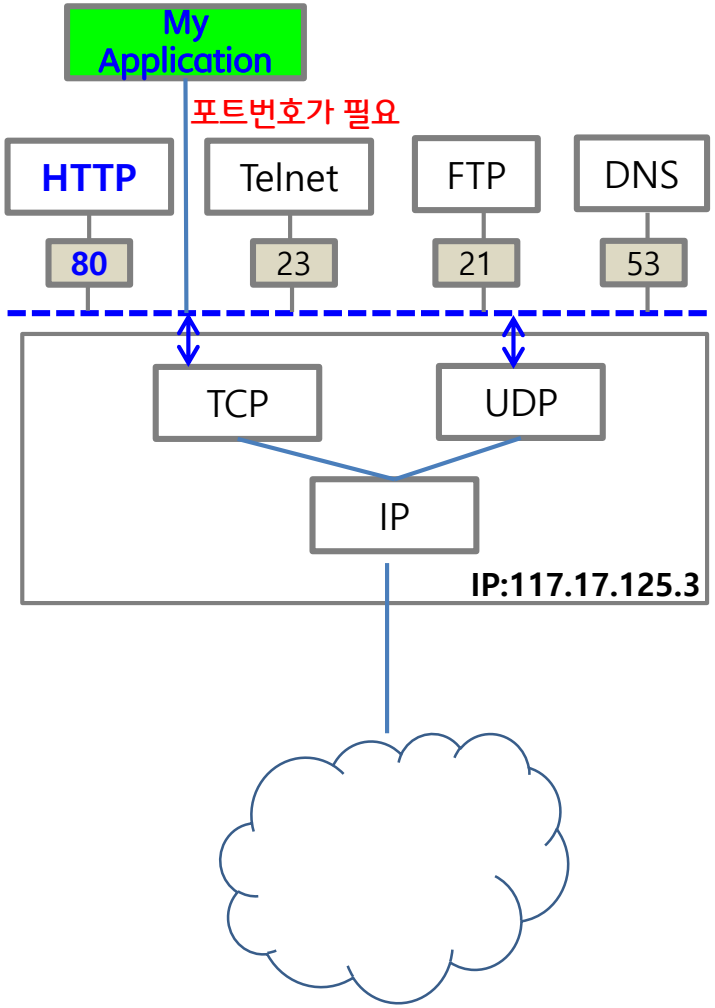
서비스	애플리케이션층 프로토콜	포트 번호	트랜스포트층 프로토콜
WWW	HTTP	80	TCP/UDP
WWW(보안 추가)	HTTPS	443	
전자메일(송신)	SMTP	25	
전자메일(수신)	POP3	110	
파일 전송	FTP	20 · 21	
원격 로그인	TELNET	23	
원격 로그인(보안)	SSH	22	
넷뉴스	NNTP	119	
네트워크 관리 DNS	DNS	53	
네트워크 관리 DHCP	DHCP	546 · 547	UDP
네트워크 관리 SNMP	SNMP	161 · 162	

2) 예약된 포트 (Registered port) : 1024 ~ 49151

*사용자가 직접 등록해서 사용할 수 있는 포트 (사용자포트)
클라이언트에서 사용할 수 있는 소켓 포트

3) 동적, 사설 포트 : 49152 ~ 65535

* 시스템에서 동적으로 할당하는 포트



2. TCP 소켓 절차 - 함수

■ 소켓의 타입 (통신방식: 전송계층 프로토콜)

```
SOCK_STREAM      /* 연결형 서비스, TCP */
SOCK_DGRAM       /* 비연결형 서비스, UDP */
SOCK_RAW
```

※ SOCK_RAW : IP 프로토콜을 직접 사용해 통신할 때 사용함

- 소켓 생성시 소켓의 타입을 지정 - 사용할 IP주소 체계와 통신방식 지정

서버 TCP 소켓 생성의 예

import socket

IP주소 형식 통신방식

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

- 첫 번째 인자(Address Family): 주소 체계를 지정하는 인자 (default는 IPv4)

AF_INET = IP4v , AF_INET6 = IP6v

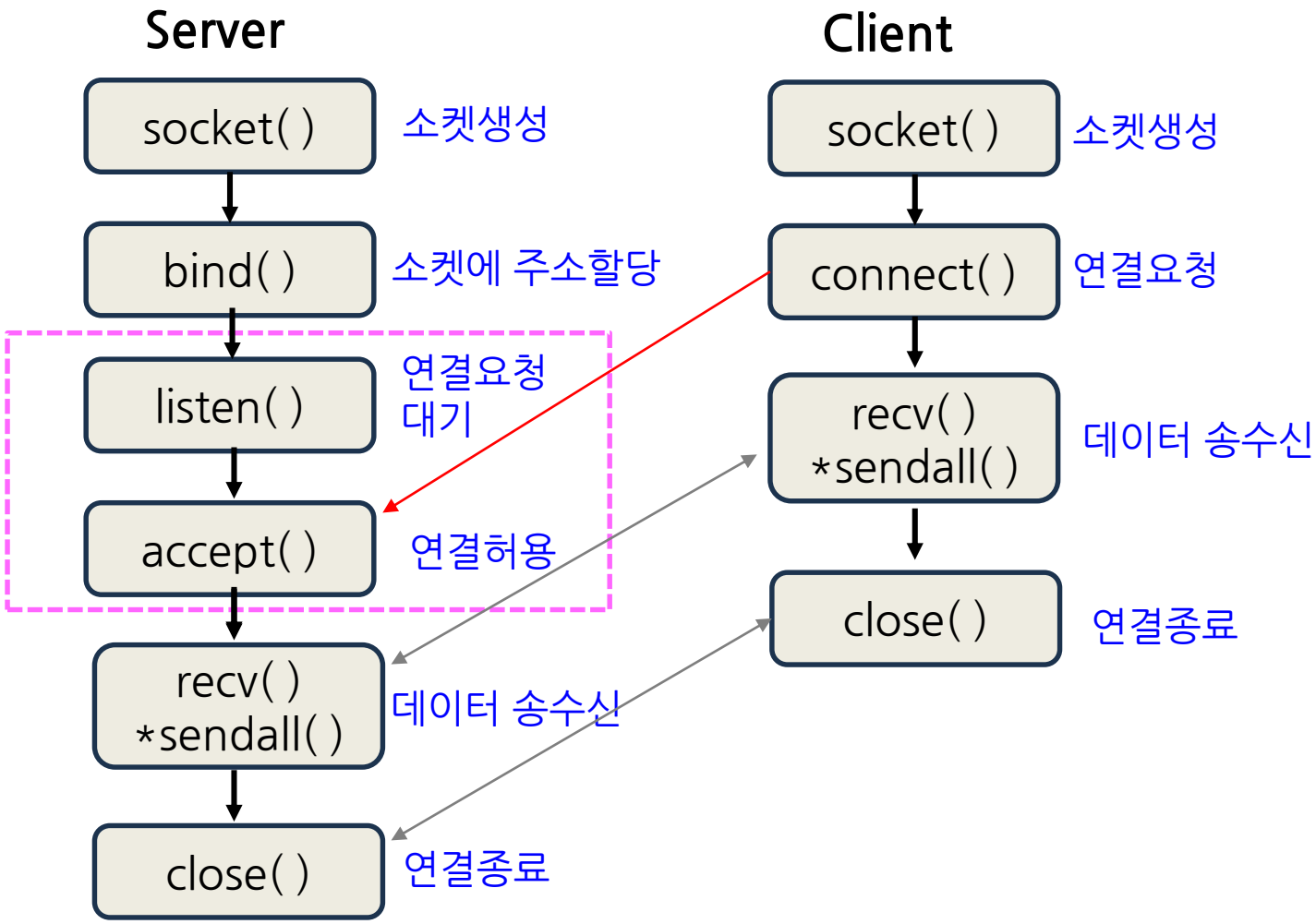
- 두 번째 인자(type): 소켓의 타입 (default는 TCP)

SOCK_STREAM = TCP , SOCK_DGRAM = UDP

2. TCP 소켓 절차 - 함수

■ 소켓 프로그래밍 절차

① 소켓생성 ②연결 설정 ③데이터 송수신 ④연결해제 순으로 수행



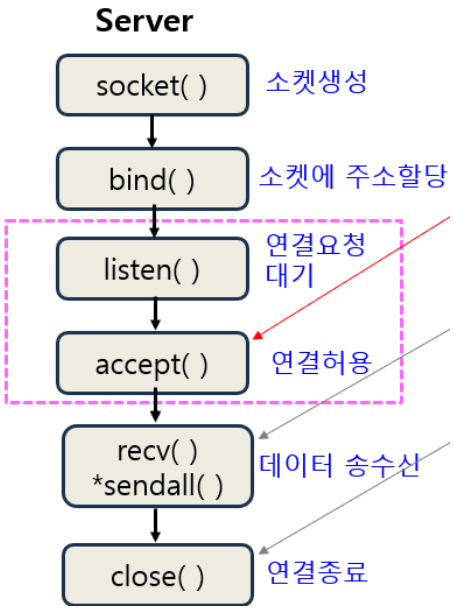
** 파이썬에서 소켓으로 메시지 보낼 때는 기존 send( ) 대신에 sendall( ) 사용 가능*
sendall( )은 파이썬 기반의 함수로 버퍼내 모든 데이터를 전송하는 기능제공

2. TCP 소켓 절차 - TCP 에코 서버와 클라이언트 프로그래밍

▪ TCP 에코 서버 프로그램

tcpEchoServer.py

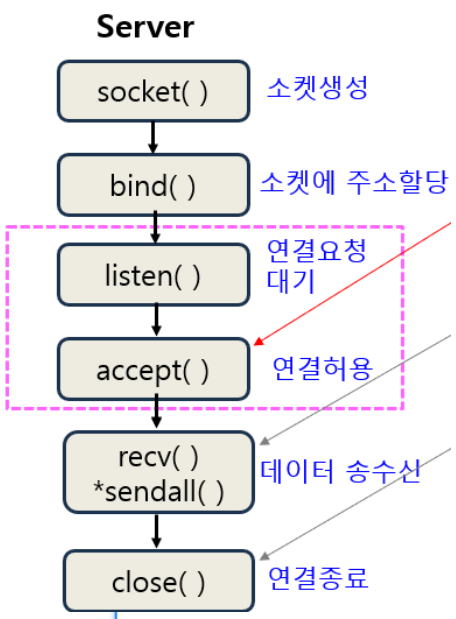
```python
# TCP Echo server program

import socket

IP = 'localhost'  # loopback ip (127.0.0.1)
PORT = 9999
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo server start")

# socket() 서버 소켓 생성
# AF_INET:주소종류는 IPv4, SOCK_STREAM:통신타입은 TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# (optional code) winError 10048 에러 해결을 위한 추가 코드
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)

# bind(주소정보) 소켓에 IP주소와 포트번호 지정
server.bind(address)
# listen() 클라이언트 접속요청 대기 상태
server.listen()
print(">> Echo server wait.........")

```



2. TCP 소켓 절차 - TCP 에코 서버와 클라이언트 프로그래밍

▪ TCP 에코 서버 프로그램

```
24 # accept() 클라이언트 접속 요청이 있을 시 수락
25 # 통신소켓과 클라이언트 주소정보를 반환
26 conn, addr = server.accept()
27 print(f">> {addr} is connected")
28
29 # recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
30 # 통신소켓에 데이터가 들어오면 메시지 반환
31 rxmsg = conn.recv(1024)
32 msg = rxmsg.decode('utf-8')  # 받을때는 디코딩
33 print(">> recv msg:", msg)
34
35 # sendall(보낼데이터) 통신소켓에 메시지 보내기
36 # 에코서버는 받은 데이터를 그대로 돌려줌
37 conn.sendall(msg.encode('utf-8'))  # 보낼때는 인코딩
38
39 # close() 서버 소켓 종료
40 conn.close()      # 통신소켓 종료
41 server.close()    # 리스닝소켓 종료
42
43 print(">> Echo server End")
```

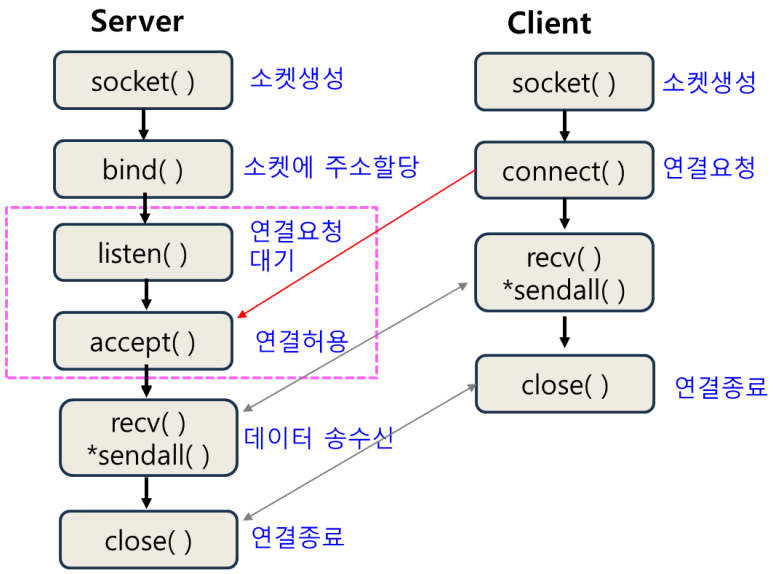


2. TCP 소켓 절차 - TCP 에코 서버와 클라이언트 프로그래밍

■ TCP 에코 클라이언트 프로그램

tcpEchoClient.py

```python
# TCP echo client program

import socket

IP = 'localhost'  # 서버 IP
PORT = 9999       # 서버 포트번호
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo client start")

# socket() 클라이언트 소켓 생성 (ipv4, tcp 소켓)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect(서버주소정보) 서버소켓으로 연결요청
client.connect(address)
print(">> Connect server")

# sendall(보낼데이터) 서버로 메시지 보내기
msg = input(">> send msg: ")
client.sendall(msg.encode('utf-8')) # 보낼 때 인코딩

# recv(받을 메시지크기) 에코서버로부터 되돌아 온 메시지 수신
rxmsg = client.recv(1024)
msg = rxmsg.decode('utf-8')   # 받을때는 디코딩
print(">> recv msg: ", msg)

# close() 클라이언트 소켓 종료
client.close()
print(">> Client End")
```



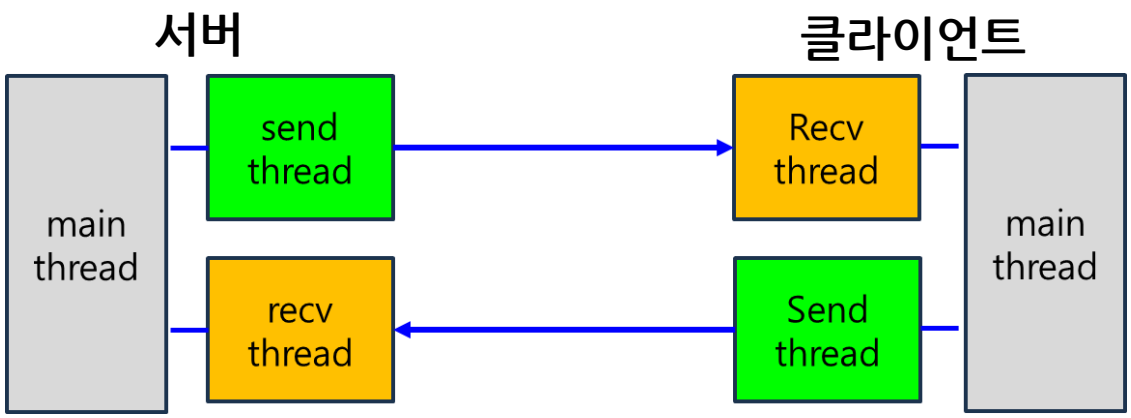
3. 스레드와 TCP 소켓을 이용한 1:1 채팅 - 스레드 개념

■ 프로세서와 스레드

- 프로세스(Process): 실행중인 프로그램
- 스레드(Thread): 프로세스 안에서 병렬로 실행중인 하위 작업 단위
- 기본적으로 파이썬은
  프로세스는 기본적으로 하나의 싱글 스레드에서 동작 : 메인 스레드(main thread)
  메인 스레드가 코드를 순차적으로 실행

- 멀티 스레드를 이용한 병렬 실행

- 멀트 스레드란 하나의 프로세스에서 둘 이상의 스레드가 동시에 작업을 수행하는 것
  → 병렬 실행 (Multi Threading)



- 네트워크에서 스레드를 사용함으로써 데이터 송신과 수신을 병렬로 실행이 가능

3. 스레드와 TCP 소켓을 이용한 1:1 채팅 - 스레드 사용법(서버)

- 스레드와 TCP 소켓을 이용한 1:1 채팅 서버 프로그램 (파일: th_tcpChatServer.py)

```python
# TCP chatting server program w/ thread

import socket, threading

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode()
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Client, please 'exit'...")
            break
        print("Client >>", rxmsg)

def send(socket):
    while True:
        sendmsg = input("Server >> ")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print("Exit by User")
            break

if __name__ == "__main__":
    IP = 'localhost'  # loopback ip (127.0.0.1)
    PORT = 9999
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Server start")
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # (optional code) winError 10048 에러 해결을 위한 추가 코드
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)
    server.bind(address)
    server.listen()
    print(">> Server wait.........")
    conn, addr = server.accept()
    print(f">> {addr} is connected")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(conn,))
    receiver.start()
    sender = threading.Thread(target=send, args=(conn,))
    sender.start()
    while True:
        receiver.join()
        sender.join()
        break
    conn.close()      # 통신소켓 종료
    server.close()      # 리스닝소켓 종료
    print(">> Server End")
```

join( )을 사용하면 서브스레드 종료후에 메인 스레드 처리가능

3. 스레드와 TCP 소켓을 이용한 1:1 채팅 - 스레드 사용법(클라이언트)

- 스레드와 TCP 소켓을 이용한 1:1 채팅 클라이언트 프로그램(파일: th_tcpChatClinet.py)

```python
# TCP chatting client program w/ thread

import socket, threading

def send(socket):
    while True:
        sendmsg = input("Client >>")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print(">> Exit by User")
            break

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode('utf-8')
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Server, please 'exit'...")
            break
        print("Sever >>", rxmsg)

if __name__ == "__main__":
    IP = 'localhost'  # 서버 IP
    PORT = 9999       # 서버 포트번호
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Client start")
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(address)
    print(">> Connect server")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(client,))
    receiver.start()
    sender = threading.Thread(target=send, args=(client,))
    sender.start()

    while True:
        receiver.join()
        sender.join()
        break

    client.close()
    print("Client End")
```

join( )을 사용하면 서브스레드 종료후에 메인 스레드 처리가능

4. UDP 소켓 - 특징

▪ UDP 소켓 서버/클라이언트 특징

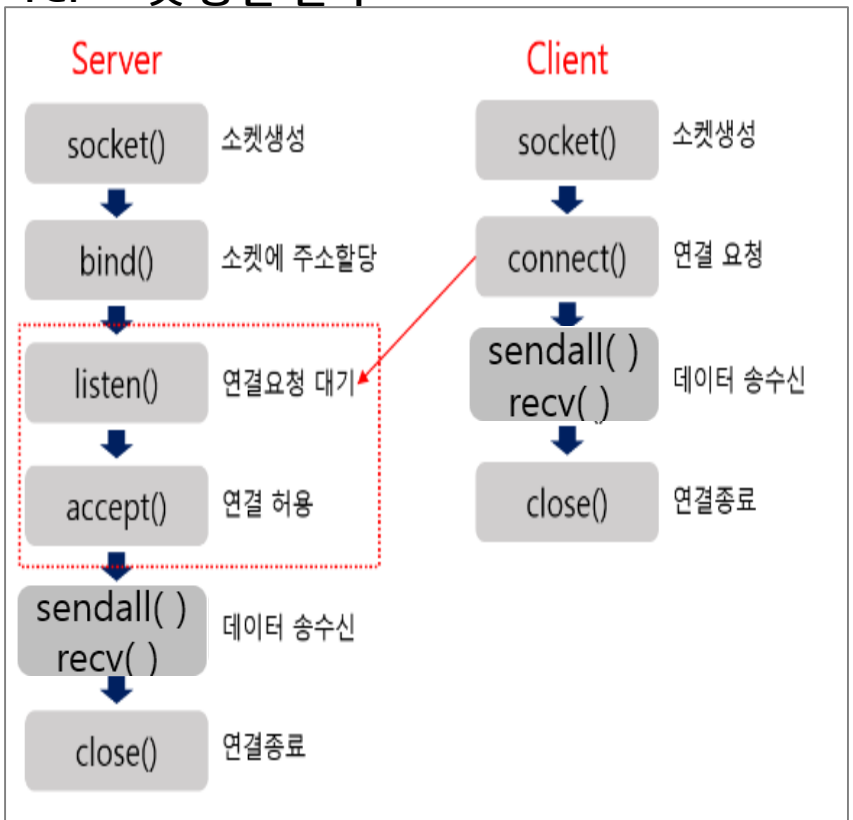
1) UDP 에서의 서버와 클라이언트는 연결되어있지 않다.

- TCP와 달리 연결 설정의 과정이 필요 없다.

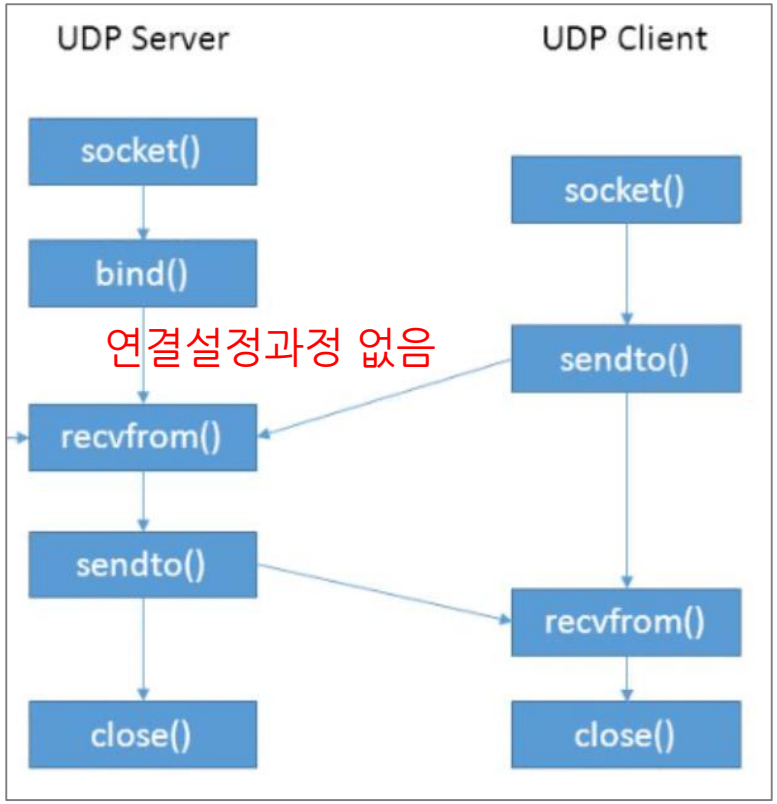
  따라서 TCP 서버 구현과정에서 거쳤던 listen 함수와 accept 함수의 호출은 없다

  다만 UDP 소켓의 생성과 데이터의 송수신 과정만 존재

TCP 소켓 통신 절차



UDP 소켓 통신 절차



4. UDP 소켓 - 특징

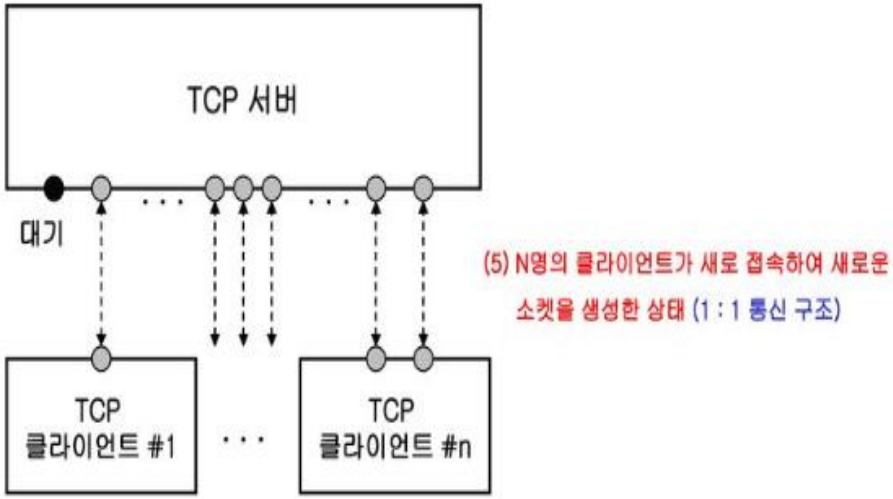
▪ UDP 소켓 서버/클라이언트 특징

2) UDP에서는 서버와 클라이언트 모두 하나의 소켓만 있으면 된다.

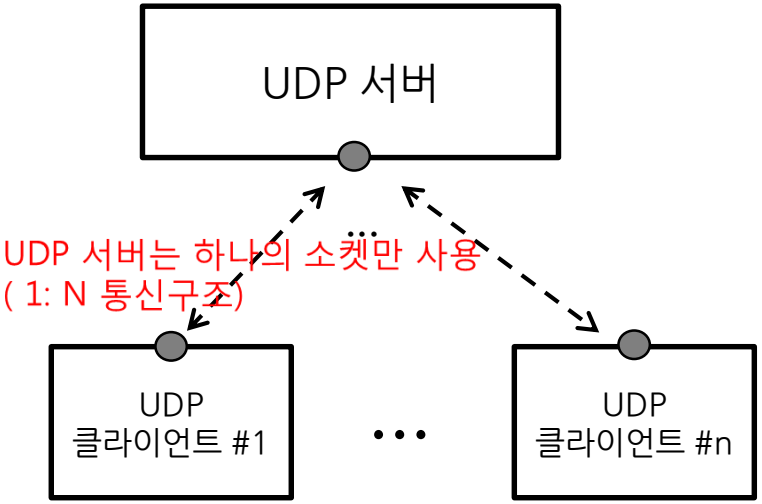
예) 10개의 클라이언트와 통신하기 위해서

- TCP에서는 소켓과 소켓의 관계가 1:1이었다.
  때문에 서버에서 열 개의 클라이언트에게 서비스를 제공하려면
  서버(리스닝) 소켓 하나와 열 개의 통신소켓이 더 필요
- 그러나 UDP는 서버건 클라이언트건 하나의 소켓만 있으면 된다

TCP 소켓 구조



UDP 소켓 구조



4. UDP 소켓 - 특징

■ UDP 소켓 서버/클라이언트 구현

3) UDP 소켓에서 데이터 전송 시 상대방의 주소정보 별도 추가

- **TCP 소켓**을 생성하고 나서 데이터를 전송하는 경우,
  데이터 전송 시 상대방의 주소 정보를 따로 추가하는 과정이 필요 없다.
  (TCP 소켓은 연결설정을 통해 이미 목적지의 주소정보를 이미 알고 있는 상태이기 때문)

TCP

전송
```
client.sendall("Hello Server".encode('utf-8'))
```

수신
```
rxmsg = conn.recv(1024)
```

- 그러나 **UDP 소켓**은 연결상태를 유지하지 않으므로
  데이터를 전송할 때마다 반드시 목적지의 주소 정보를 별도로 추가해야 한다.
  수신 시에는 return 값이 두 개 (수신메시지와 상대방의 주소 정보)

상대방 목적지의 IP주소와 포트정보 추가

UDP

전송
```
client.sendto("Hello Server".encode(), ('10.2.100.34', 8888))
```

수신 시 return값이 두 개 (수신메시지, 상대방의 IP주소와 포트정보)

수신
```
data, addr = server.recvfrom(1024)
```

4. UDP 소켓 - 함수

- **UDP 소켓 서버 함수**

서버소켓 생성

server = socket.**socket**(socket.AF_INET, socket.SOCK_DGRAM)

Family = IPv4 Type = UDP

서버소켓 바인딩

server.**bind** ((**서버IP주소**, **PORT번호**)) #튜플로 저장

서버 소켓에서 메시지 수신

data, addr = server.**recvfrom** (1024) **# BUFFER_SIZE = 1024**

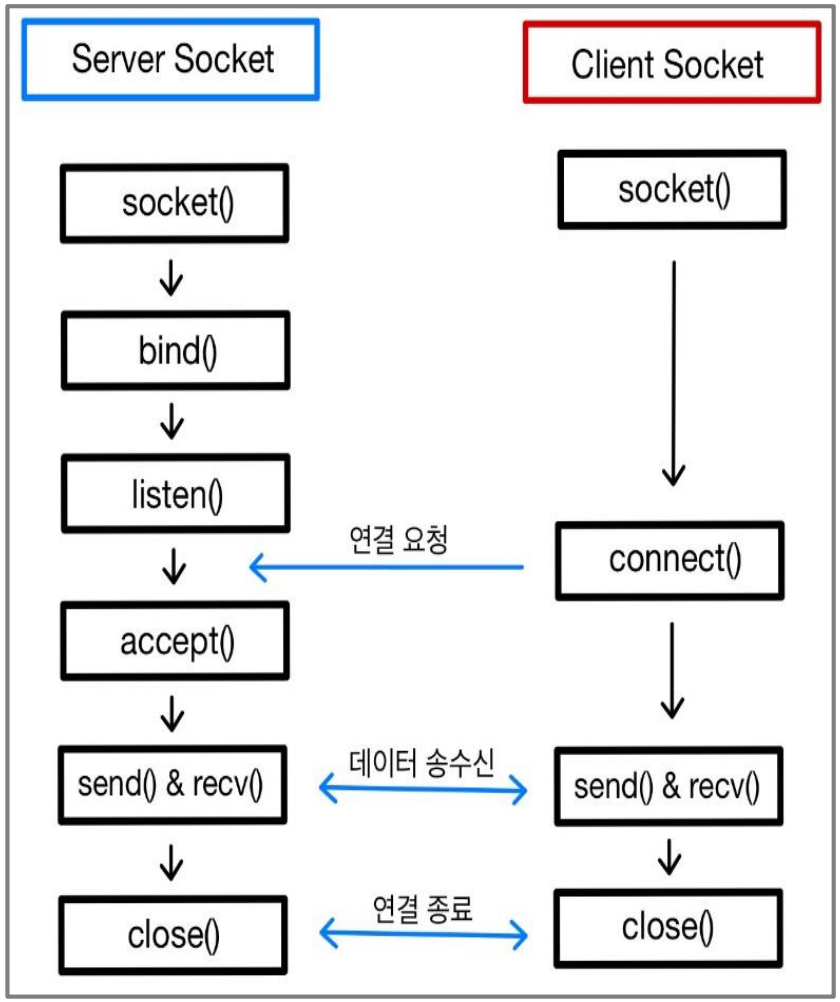
상대방의 IP주소와 포트정보

서버 소켓에서 메시지 전송

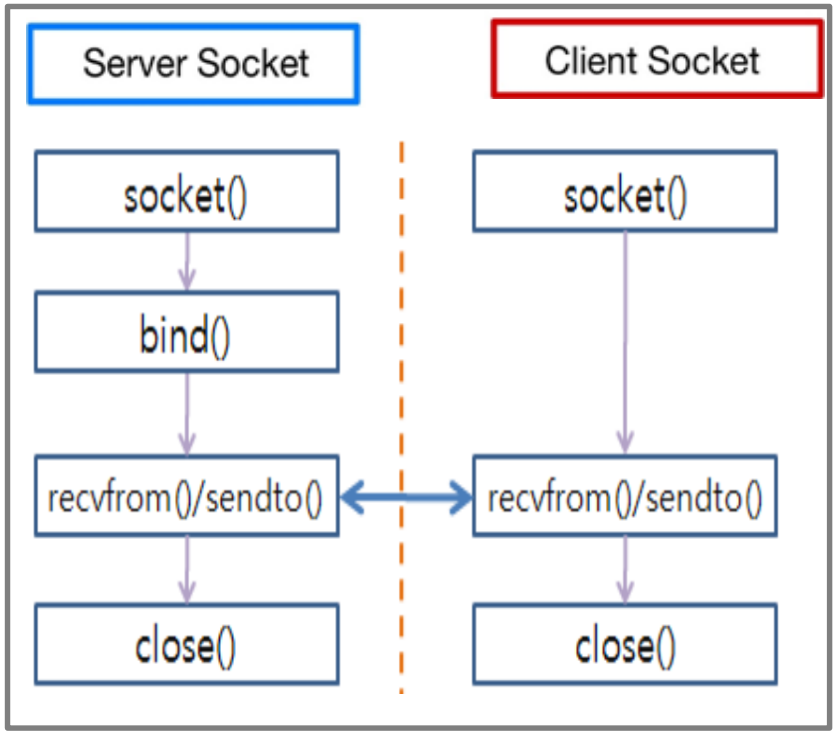
server.**sendto** (data.encode( ), (**목적지IP주소**, **목적지PORT번호**))

TCP 소켓과 UDP 소켓의 통신 절차 비교

TCP 소켓통신 절차

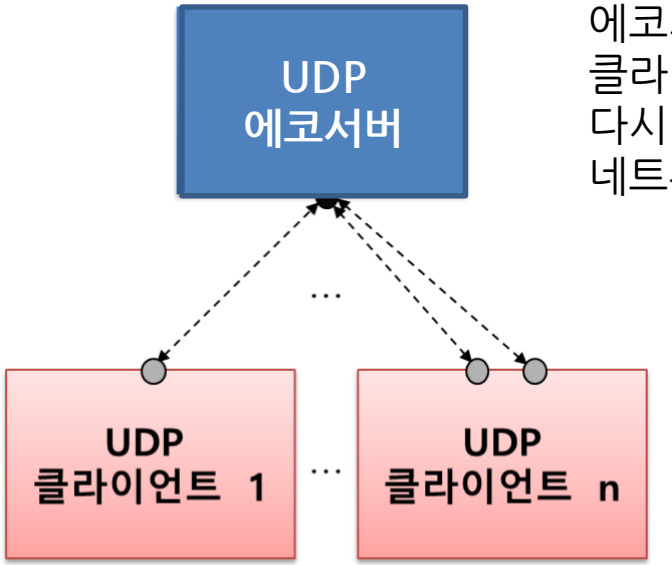


UDP 소켓통신 절차



- 연결설정과정이 없음
- 서버는 UDP 소켓 1개만 사용
- 데이터 전송 시 상대방의 주소정보 추가
- 데이터 수신 시 상대방의 주소정보 반환

UDP 소켓을 이용한 에코서버와 클라이언트



에코서버란?
클라이언트가 보낸 데이터를 서버에서 받아 그대로 다시 클라이언트에게 돌려보내 주는(메아리) 기본적인 네트워크 서버

- 스레드를 사용하지 않고도, 한 소켓으로 여러 클라이언트 처리 가능
- 클라이언트는 별도의 연결설정 과정없이 서버와 통신
- UDP 서버는 하나의 소켓을 이용해 다수의 클라이언트와 통신 가능

4. UDP 소켓 - UDP 소켓 프로그램 (에코서버와 클라이언트)

■ UDP 소켓을 이용한 에코서버와 클라이언트

```python
# UDP 소켓을 이용한 에코 서버 프로그램
import socket

IP = 'localhost'  # localloop ip
PORT = 9000
address = (IP, PORT)

print(">> UDP Echo Server start")
# UDP 소켓 객체 생성
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# 소켓 binding: 소켓에 IP주소와 port번호 지정
server.bind(address)

while True:
    # recvfrom(): 클라이언트 메시지 수신
    data, addr = server.recvfrom(1024)
    print(f"{addr}:{data.decode('utf-8')}")

    # sendto(data,addr): 클라이언트가 보낸메시지 그대로 전송
    server.sendto(data, addr)

server.close()
print(">> UDP Server End")
```

```python
# UDP 소켓을 이용한 에코서버 클라이언트 프로그램
import socket

IP = 'localhost'  # localloop IP
PORT = 9000
address = (IP, PORT)

print(">> UDP Echo Client Start")
# UDP 소켓 객체 생성
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
    # 보낼메시지 입력
    sendmsg = input("Send:")
    # sendto() 서버로 메시지 보내기
    if sendmsg == 'exit':
        print("Disconnect by User")
        break
    client.sendto(sendmsg.encode('utf-8'), address)
    # recvfrom() 서버로부터 되돌아 온 메시지 수신
    data, addr = client.recvfrom(1024)
    print("Recv:", data.decode('utf-8'))

client.close()
print(">> UDP client End")
```

Thank You